

PAPER • OPEN ACCESS

Typical machine learning datasets as low-depth quantum circuits

To cite this article: Florian J Kiwit *et al* 2025 *Quantum Sci. Technol.* **10** 045035

View the [article online](#) for updates and enhancements.

You may also like

- [Quantum machine learning for image classification](#)
Arsenii Senokosov, Alexandr Sedykh, Asel Sagingalieva et al.
- [Development of hybrid quantum classifiers for realistic classification tasks](#)
Sara Aminpour, Sarah Sharif and Mike Banad
- [Performance analysis of quantum convolutional layers for image classification](#)
Qiu-Yu Xiang, Dong-Fen Li, Yu-Chen Sun et al.

Quantum Science and Technology



PAPER

OPEN ACCESS

RECEIVED
19 May 2025

REVISED
12 August 2025

ACCEPTED FOR PUBLICATION
27 August 2025

PUBLISHED
15 September 2025

Original Content from
this work may be used
under the terms of the
[Creative Commons
Attribution 4.0 licence](#).

Any further distribution
of this work must
maintain attribution to
the author(s) and the title
of the work, journal
citation and DOI.



Typical machine learning datasets as low-depth quantum circuits

Florian J Kiwit^{1,2,*} , Bernhard Jobst^{3,4,*} , Andre Luckow^{1,2} , Frank Pollmann^{3,4}
and Carlos A Riofrío²

¹ Ludwig Maximilian University, Munich, Germany

² BMW Group, Munich, Germany

³ Physics Department, Technical University of Munich, TUM School of Natural Sciences, 85748 Garching, Germany

⁴ Munich Center for Quantum Science and Technology (MCQST), Schellingstr. 4, 80799 München, Germany

* Authors to whom any correspondence should be addressed.

E-mail: florian.kiwit@ifi.lmu.de and bernhard.jobst@tum.de

Keywords: quantum algorithms, quantum computation, machine learning, variational quantum circuits

Abstract

Quantum machine learning (QML) is an emerging field that investigates the capabilities of quantum computers for learning tasks. While QML models can theoretically offer advantages such as exponential speed-ups, challenges in data loading and the ability to scale to relevant problem sizes have prevented demonstrations of such advantages on practical problems. In particular, the encoding of arbitrary classical data into quantum states usually comes at a high computational cost, either in terms of qubits or gate count. However, real-world data typically exhibits some inherent structure (such as image data) which can be leveraged to load them with a much smaller cost on a quantum computer. This work further develops an efficient algorithm for finding low-depth quantum circuits to load classical image data as quantum states. To evaluate its effectiveness, we conduct systematic studies on the MNIST, Fashion-MNIST, CIFAR-10, and Imagenette datasets. The corresponding circuits for loading the full large-scale datasets are available publicly as PennyLane datasets and can be used by the community for their own benchmarks. We further analyze the performance of various quantum classifiers, such as quantum kernel methods, parameterized quantum circuits, and tensor-network classifiers, and we compare them to convolutional neural networks. In particular, we focus on the performance of the quantum classifiers as we introduce nonlinear functions of the input state, e.g. by letting the circuit parameters depend on the input state.

1. Introduction

Machine learning is a powerful framework that has facilitated tremendous progress in problems such as image recognition and text generation, which would have been considered intractable using conventional programming. Quantum computers promise to enhance the capabilities of machine learning models further, generating a significant amount of interest in the field of quantum machine learning (QML) [1, 2]. Indeed, several theoretical works suggest an exponential advantage of QML over classical machine learning methods for certain specific tasks [3–7], such as solving a synthetic classification task constructed from partitioning the data based on the discrete logarithm [3]. These tasks tend to be quite contrived from an application-based view, owing to the difficulty of making more natural tasks amenable to mathematical proofs [8]. Hence, the demonstration or existence of the advantages mentioned above is still an important open question [9].

A key challenge is the loading of classical datasets as quantum states. On the one hand, popular data encoding methods include *rotation encoding* [1, 10], which uses a single qubit per data component and rotates the state of the corresponding qubit proportional to the data component. The encoding is popular because it is very efficient to implement as a circuit and allows to prove universality results based on data reuploading techniques [11–13]—however, the number of qubits for relevant datasets quickly becomes of the order of millions, which are not available in the near term.

On the other hand, there are popular techniques like *amplitude encoding* [1, 14], which only need a number of qubits logarithmic in the number of data components by storing the (normalized) values in the amplitudes of the wave function. In general, however, preparing such a state requires a number of gates that scales exponentially with the number of qubits, rendering the method inefficient and unrealistic for the gate fidelities and coherence times of the currently available quantum computers [15, 16].

This data-loading problem has so far limited many demonstrations of QML to small-scale datasets in both experiments and numerical simulations, often utilizing either artificial or simple datasets, or applying dimensionality reduction methods like the principal component analysis or coarse-graining [17–29]. While many QML models achieve promising results on simple datasets like MNIST and Fashion-MNIST, the same models tend to fail badly on more complex and realistic datasets like CIFAR-10 [30–34]. This highlights the need to test QML models on relevant realistic datasets as a good performance on simple datasets does not necessarily translate to more complicated datasets.

Further, the use of many different datasets—unlike the common benchmarking datasets widely used in classical machine learning—makes it difficult to compare the performance of different models directly. Such standard datasets have been highly successful in classical machine learning. One example from the early development of modern neural networks is the ‘ImageNet Large Scale Visual Recognition Challenge’ [35], in which AlexNet [36] first demonstrated the success of deep-learning architectures.

In the field of QML, efforts are now being undertaken to compare the performance of QML models on common datasets or using standardized benchmarks [29, 37–42]. In a recent study, [29] compared many popular QML models on several small-scale and mostly artificial datasets, and concluded that classical models consistently outperform their quantum counterparts. As already raised by the study’s authors, an interesting open question is whether this observation also holds true for more complicated or real-world datasets.

Moreover, a step towards real-world application-oriented benchmarking is taken by the creation of software tools that provide a modular and standardized pipeline to evaluate models [39–42]—however, examples like the QUARK framework [39–41] focus on optimization problems and generative QML, while the MQT benchmark library [42] focuses on general quantum computing tasks. Nevertheless, there is still a lack of options for assessing the scalability and the capability of supervised QML models to deal with complex real-world data.

In this work, our main goal is to address the lack of real-world, large-scale datasets for benchmarking QML models, with a focus on keeping the hardware requirements to a minimum so that the datasets can be relevant for near-term experiments and numerical simulations. Inspired by the recent success of approximating amplitude-encoding-type states with tensor networks and shallow quantum circuits [43–47], we devise and implement an algorithm for efficiently optimizing such quantum circuits on a classical computer. This yields quantum circuits whose number of qubits scales only logarithmically in the input data size and whose depth only scales linearly in the number of qubits. First, we apply this algorithm to image data, specifically the MNIST [48, 49], Fashion-MNIST [50], CIFAR-10 [51] and Imagenette [52] datasets, and provide the encoded images as PennyLane datasets [53] at <https://pennylane.ai/datasets/collection/low-depth-image-circuits>.

While our main focus is on obtaining these datasets, we also benchmark variational quantum circuit classifiers, support vector machines, and tensor-network classifiers on them and find that these models show promising results on the simple MNIST and Fashion-MNIST datasets but struggle on the harder and more realistic CIFAR-10 and Imagenette datasets. We further propose strategies to turn the quantum circuit classifiers from a linear into a nonlinear function of the input density matrix. While these strategies in their current form only enhance the accuracy of the classifier by several percentage points, which is still not enough to make them competitive with current classical state-of-the-art methods, we view them as potentially fruitful directions for further exploration. We invite researchers to try to reach state-of-the-art performance by using their own ideas for quantum classifiers on these datasets.

This paper is structured as follows: In section 2, we review the methods we use for encoding classical image data into quantum states. Building on this encoding, in section 3 we propose a low-depth quantum circuit architecture inspired by matrix-product states and present an algorithm that efficiently optimizes these circuits to approximate the encoded states on a classical computer. We show results on the MNIST, Fashion-MNIST, CIFAR-10, and Imagenette datasets, demonstrating the approach’s efficacy. In section 4, we benchmark various quantum classifiers for image classification on these datasets, including variational quantum circuits (VQCs), a nonlinear variant of VQCs, support vector machines (SVMs) with a quantum kernel and tensor-network classifiers. First, we consider the exact dataset without the circuit approximation, then we assess the effect of the circuit approximation on the classification accuracy by also training the classifiers on the compressed dataset. We further compare the performance on the compressed dataset to that

of a classical convolutional neural network (CNN) trained on the same data. In section 5, we conclude with a summary of our results.

2. Quantum image representations

Our main goal is to construct quantum states encoding classical data from realistic and large-scale datasets, that also efficiently use available quantum resources—both in terms of the number of qubits used and the depth of the circuit needed to prepare the corresponding state.

An encoding that minimizes the circuit depth is, for example, the *rotation encoding* [1, 10], where each qubit encodes one component of the input data vector by rotating its state proportional to this data component, e.g. by applying a Pauli rotation gate $e^{i\sigma_j^\alpha x_j}$ where σ_j^α is a Pauli matrix acting on the j th qubit and x_j is the j th data component. The single-qubit gates can all be applied in parallel and no entangling operations are needed, making the state preparation very efficient. However, this comes at the cost of requiring a large number of qubits: to encode moderate-sized images with 512×512 pixels and the three RGB color channels, we need over 750 000 qubits. Suppose a subsequent QML model needs to correlate all pixel values to make a correct prediction, then the total gate count will reach a similar order of magnitude. This makes the approach infeasible for experimental realizations for a long while.

On the other hand, approaches like *amplitude encoding* minimize the number of qubits needed [1, 14]. Encoding data in the amplitudes of a superposition of basis states allows, in principle, to encode an exponential number of data components in a linear number of qubits. In the previous example, this reduces the qubit requirements from over 750 000 to just about 20 qubits. The caveat now being that preparing arbitrary data vectors using this encoding becomes exponentially costly. However, actual data of interest is often not arbitrary but follows some internal structure, e.g. how images differ from random pixel values. In fact, for typical images there have been several recent results (both numerical and theoretical) showing that their underlying structure leads to quantum states after the encoding that are well-captured by tensor-network states and by tensor-network-inspired quantum circuits [43–47]. They can thus be prepared in a depth linear in the number of qubits required for the encoding. While we focus on image data in this paper, there are other types of data which result in lowly entangled states after amplitude encoding, for which we expect the methods of this paper to work similarly well [44, 54–58].

Motivated by these results, we consider similar encodings in the following. We will also consider a way to interpolate between the amplitude-type encoding and the rotation encoding [43]—as the rotation encoding is often considered a more powerful nonlinear feature map for a subsequent machine learning model [1, 10]. Specifically, we consider two encodings for grayscale and color images that result in quantum states of the form

$$|\psi(\mathbf{x})\rangle = \frac{1}{\sqrt{2^n}} \sum_{j=0}^{2^n-1} |c(\mathbf{x}_j)\rangle \otimes |j\rangle \quad (1)$$

for images with 2^n pixels [59]. The state $|j\rangle$ of the n so-called *address qubits* tracks the position index j of the pixel in the image. The state $|c(\mathbf{x}_j)\rangle$ encodes the data value $\mathbf{x}_j$ of the j th pixel. For grayscale images this encodes the grayscale value in a single *color qubit*, for color images this encodes the vector of RGB values using three color qubits. Similarly to amplitude encoding, these image encodings only require a number of qubits that scales logarithmically with the number of pixels, but they have some further advantages: whereas amplitude encoding can lose contrast information due to forced normalization, the states in equation (1) circumvent that issue by directly constructing correctly normalized states; empirically, they have a slightly smaller entanglement entropy than amplitude-encoded states making them more compressible as tensor-network states [44]; and, finally, in image classification tasks states using such encodings could be classified with a slightly higher accuracy than their amplitude-encoded counterparts [46].

The order in which the pixels are indexed can change the entanglement entropy of the resulting state [44]. Here, we choose hierarchical indexing (shown in figure 1(a)) based on the so-called Z- or Morton order [14, 44, 60, 61]—where the first two bits of the index j label the quadrant of the image the pixel is in, the next two bits label the subquadrant, and so on. This tends to further decrease the entanglement entropy compared to other orderings, and thus should make the resulting states more compressible (see [44] for grayscale images and appendix A for color images).

Flexible representation of quantum images. For grayscale images, we use the *flexible representation of quantum images (FRQI)* [60, 61] as an encoding. In this case, the data value $x_j \equiv \mathbf{x}_j$ of each pixel is just a single number corresponding to the grayscale value of that pixel. We can encode this information in the z-polarization of an additional color qubit as

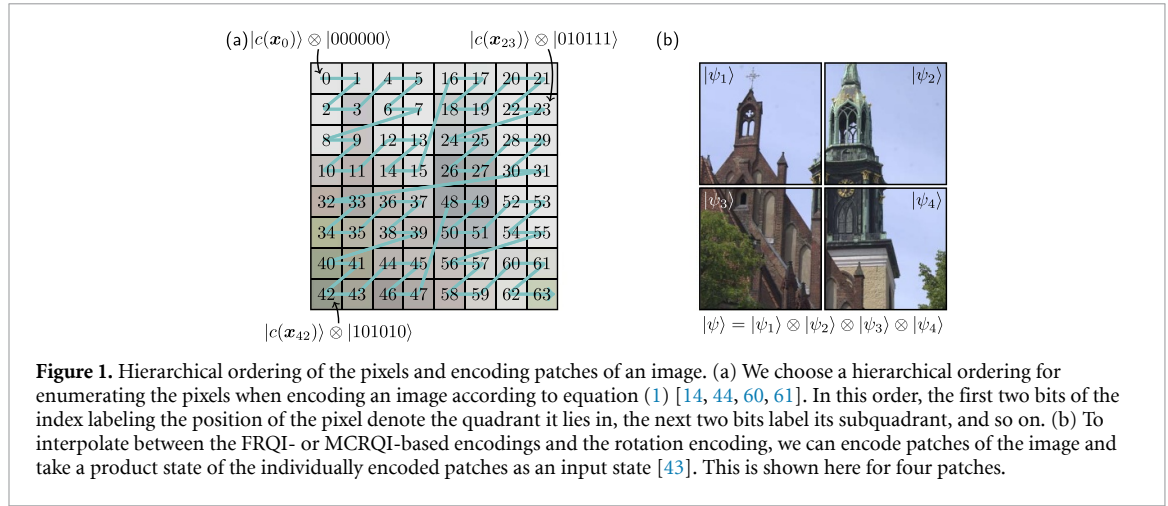


Figure 1. Hierarchical ordering of the pixels and encoding patches of an image. (a) We choose a hierarchical ordering for enumerating the pixels when encoding an image according to equation (1) [14, 44, 60, 61]. In this order, the first two bits of the index labeling the position of the pixel denote the quadrant it lies in, the next two bits label its subquadrant, and so on. (b) To interpolate between the FRQI- or MCRQI-based encodings and the rotation encoding, we can encode patches of the image and take a product state of the individually encoded patches as an input state [43]. This is shown here for four patches.

$$|c(x_j)\rangle = \cos\left(\frac{\pi}{2}x_j\right)|0\rangle + \sin\left(\frac{\pi}{2}x_j\right)|1\rangle, \quad (2)$$

with the pixel value normalized to $x_j \in [0, 1]$. Thus, a grayscale image with 2^n pixels is encoded into a quantum state with $n + 1$ qubits—see equation (1). In general, the complexity of preparing the resulting state *exactly* scales exponentially with the number of qubits. Known constructions (without auxiliary qubits) use $\mathcal{O}(4^n)$ gates [60, 61] or, allowing for classical preprocessing with $\mathcal{O}(n2^n)$ steps, construct a circuit with $\mathcal{O}(2^n)$ gates [59]. However, encoding typical images this way leads to lowly entangled quantum states that are well approximated by tensor-network states such as matrix-product states (MPSs) [43–47] whose bond dimension χ does not need to scale with the image resolution [44]. Thus, preparing the state *approximately* with a small error is possible with a number of gates that scales only as $\mathcal{O}(\chi^2 n)$, i.e. linearly with the number of qubits. Note that the cost of classical preprocessing to obtain the MPS via successive SVDs is $\mathcal{O}(\chi^2 n)$. While this is technically exponential in the number of qubits, this is not a problem in practice: the images we consider must still be stored classically. The cost of computing the MPS is only linear in the number of pixels, so the cost of compressing the image is not substantially more than storing it. In certain cases, this cost can even be further reduced to $\mathcal{O}(\chi^3 n)$ by making use of tensor cross interpolation algorithms [62–66]. In any case, while the cost of the classical preprocessing may be similar to the exact state preparation, the resulting quantum circuits are exponentially more efficient.

Multi-channel representation of quantum images. For color images, we use the *multi-channel representation of quantum images (MCRQI)* [67, 68] as an encoding. For each pixel, the data value now has several components, $\mathbf{x}_j = (x_j^R, x_j^G, x_j^B, x_j^\alpha)^T$, corresponding to the three RGB color channels and a possible fourth α channel indicating the opacity of the image. If only the three RGB channels are available for a given image (as is the case for all color image datasets considered in this work), the image is at full opacity and we can simply set the α channel to zero [68]. The color information of a pixel is then encoded in a three-qubit state as

$$\begin{aligned} |c(\mathbf{x}_j)\rangle = & \frac{1}{2} \left(\cos\left(\frac{\pi}{2}x_j^R\right)|000\rangle + \sin\left(\frac{\pi}{2}x_j^R\right)|100\rangle \right. \\ & + \cos\left(\frac{\pi}{2}x_j^G\right)|001\rangle + \sin\left(\frac{\pi}{2}x_j^G\right)|101\rangle \\ & + \cos\left(\frac{\pi}{2}x_j^B\right)|010\rangle + \sin\left(\frac{\pi}{2}x_j^B\right)|110\rangle \\ & \left. + \cos\left(\frac{\pi}{2}x_j^\alpha\right)|011\rangle + \sin\left(\frac{\pi}{2}x_j^\alpha\right)|111\rangle \right), \end{aligned} \quad (3)$$

with normalized values $x_j^R, x_j^G, x_j^B, x_j^\alpha \in [0, 1]$. Thus, a color image with 2^n pixels is encoded into a quantum state with $n + 3$ qubits—see equation (1). To prepare the state *exactly* on a quantum computer, we can essentially reuse the same circuit that prepares an FRQI state and run it for each color channel separately. Thus, known circuits for exact state preparation (without auxiliary qubits) use $\mathcal{O}(4^n)$ gates [67, 68] or, after classical preprocessing with $\mathcal{O}(n2^n)$ steps, use $\mathcal{O}(2^n)$ gates [59]. However, just as for grayscale images, encoding typical color images in this way results in lowly-entangled states, which are well approximated by tensor-network states. We numerically demonstrate this in appendix A, where we consider the MCRQI and two other color image encodings, and show that for all of them the bipartite entanglement entropy does not

grow with increasing image resolution. We choose to work with the MCRQI in the following, since it has the lowest entanglement entropy out of the three encodings. Due to their low entanglement, preparing color image states *approximately* can be done with a circuit whose number of gates scales only linearly with the number of qubits. We will demonstrate this numerically in the next section.

Interpolating to the rotation encoding. As mentioned in the introduction, the rotation encoding not only enjoys popularity in QML due to its efficient state preparation circuit, but also because from the point of view of kernel-based machine learning it implements a nonlinear function leading to a potentially more powerful feature map [1, 69]. As such, it has also been successfully used in tensor-network-based machine learning [10]. To try to keep some of the benefits of the rotation encoding while keeping the number of qubits manageable, we can interpolate between the FRQI- or MCRQI-based encoding and the rotation encoding by splitting the original 2^n -pixel image into N_p patches and encoding each patch separately [43]—this is shown for four patches in figure 1(b). The number of required qubits is then $n = N_p \cdot (\log_2(2^n/N_p) + n_c)$, with either $n_c = 1$, a single color qubit, for the FRQI or $n_c = 3$ color qubits for the MCRQI. Using this *patched encoding*, a single patch, $N_p = 1$, corresponds to encoding the full image, while using as many patches as pixels, $N_p = 2^n$, corresponds to encoding each pixel individually. For the FRQI, this recovers the rotation encoding exactly, with each qubit being rotated as $e^{-i\frac{\pi}{2}\sigma_j^x x_j} |0\rangle = \cos(\frac{\pi}{2}x_j) |0\rangle + \sin(\frac{\pi}{2}x_j) |1\rangle$. For the MCRQI this leads to a product state of three-qubit states encoding the RGB values of each pixel separately. By varying the number of patches N_p , we can gradually interpolate between the two limiting cases of encoding the entire image into a single entangled state or representing it as a product state of individual pixels.

Multiple copies of the encoding. Another popular technique in QML to make models more powerful is to start from a product state of multiple copies of the state encoding the input data. This can be seen as a particular instance of the more general data reuploading framework, based on which certain universality results for the outputs of quantum classifiers can be proved [11–13]. Another way of seeing the possible advantage of this approach is that this allows to interpret the output state of a unitary operation that entangles the different copies as a nonlinear transformation of the matrix elements of a single copy of the input density matrix [70]. Thus, the *multi-copy encoding* can give the quantum model access to otherwise possibly missing nonlinearities required for solving many tasks.

3. Optimizing quantum circuits for efficient state preparation

In the preceding section, we introduced encodings of classical image data that, at least for typical images, result in states with low entanglement that are effectively represented by tensor-network states. In this section, we are concerned with finding efficient circuits that (approximately) prepare those states on a quantum computer.

The efficient state preparation is possible because images have an internal structure that can be exploited for further compression. For instance, typical images exhibit a concentrated Fourier spectrum as the low-frequency components often already faithfully capture the subject and high-frequency components are perceived as noise. Truncating this Fourier spectrum leads to a representation as an MPS, whose bond dimension does not need to grow with increasing image resolution to keep the approximation error below some threshold [44]. This MPS representation of the state then allows for preparation on a quantum computer with a number of gates that scale only linearly with the number of qubits. While an MPS can be directly mapped to a linear-depth quantum circuit employing multi-qubit gates [55, 71–75], we can further reduce the required number of gates by choosing an MPS-inspired circuit architecture consisting of two-qubit gates only [43–47]. The parameters in the circuit are subsequently optimized to maximize the fidelity with the target state. We will introduce the specific circuit architecture and our optimization algorithm in the following.

3.1. Circuit architecture

A widely used quantum circuit architecture consists of multiple layers of two-qubit gates sequentially arranged in a staircase pattern [43–46, 74–78], shown graphically in figure 2 on the left. This pattern is inspired by the right-canonical form of MPSs [79, 80]. Indeed, an MPS with a bond dimension of two corresponds exactly to a single layer of such a sequential circuit, since in right-canonical form all MPS tensors are isometric and can be implemented by two-qubit gates—for larger bond dimensions, multi-qubit gates are needed for this correspondence [55, 71–75]. To avoid having to decompose multi-qubit gates, the more popular approach is to simply stack several layers of the circuit corresponding to an MPS with a bond dimension of two.

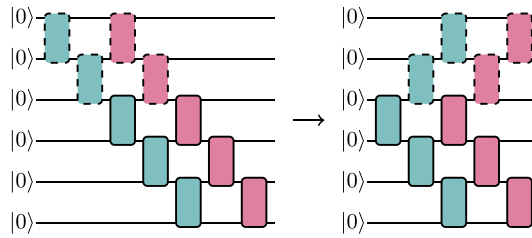


Figure 2. Illustration of quantum circuits inspired by MPSs. The left side shows a circuit with a staircase pattern with two layers (represented in turquoise and pink), where two-qubit gates are applied sequentially, corresponding to a right-canonical MPS. The right side shows the proposed circuit architecture corresponding to an MPS in mixed canonical form. By effectively shifting the gates with the dashed outlines to the right, the gates are applied sequentially outward starting from the center. This reduces the circuit depth while maintaining its expressivity.

Instead of the sequential circuit obtained from a right-canonical MPS, we can also consider an MPS in mixed canonical form, where the canonical center is a central site of the MPS. For an MPS with a bond dimension of two, this now corresponds to a sequential circuit where two staircases of two-qubit gates begin in the center of the system and move outwards [81, 82]. One layer of such a circuit is shown in figure 2 on the right in turquoise, and we refer to it as a *center-sequential* circuit. Compared to the original quantum circuit, we now obtain a circuit with reduced depth but the same expressivity. To increase the expressivity of the circuit without requiring multi-qubit gates, just as for the original sequential circuit, we can simply stack more layers of the circuit (see figure 2 on the right, turquoise and pink layers).

The two-qubit gates appearing in the circuit can be parametrized in various ways, which can differ in the number of CNOTs required to implement them on quantum hardware [83]. They thus offer a trade-off in expressivity versus CNOT gate count efficiency, which might be desirable since, on many hardware platforms, CNOT gates require longer pulse sequences and exhibit higher error rates than single-qubit gates. Specifically, we consider three different two-qubit primitives: (i) arbitrary unitary matrices in $SU(4)$ requiring three CNOTs to be implemented [84], (ii) special orthogonal matrices in $SO(4)$ requiring only two CNOTs [85], and (iii) hardware-efficient gates parametrized by a single CNOT enclosed by single-qubit gates, which we will refer to as *sparse* gates [46]. We give further details on the parametrization in the description of the algorithm below.

3.2. Optimization algorithm

After fixing the structure of the circuit to the center-sequential layout, the goal is to optimize the gates in the circuit to maximize its fidelity with the target state. We assume the target state is given as an MPS, since—as discussed in the preceding section—the target states we consider are lowly entangled and their MPS representation can be computed with acceptable cost. The optimization of the circuit then consists of four steps:

- (1) We initialize the gates in the circuit via an approximate *analytical decomposition* of the target state [76].
- (2) By *sweeping* through the circuit, we iteratively replace individual two-qubit gates with the ones maximizing the fidelity [74, 86].
- (3) We then *decompose the two-qubit gates* into CNOTs and single-qubit gates [83–85].
- (4) Finally, we further optimize the parametrized single-qubit gates using the *Broyden–Fletcher–Goldfarb–Shanno (BFGS) algorithm* [87].

This combines previous findings that the analytic initialization of the gates followed by the sweeping algorithm yields high-fidelity quantum circuits [77, 78], and that switching from an optimization algorithm with local updates like the sweeping algorithm to a global, second-order quasi-Newton method like BFGS in the latter stages of the optimization can speed up convergence and further improve the quality of the optimized state significantly [88]. In the following, we describe each algorithm step in more detail.

Analytical decomposition. When optimizing a quantum circuit, it has been shown that it is important to initialize the parameters close to an optimum to aid in convergence [38, 77, 89–95]. In our case, since the target state is given as an MPS, we can make use of the correspondence between one layer of the circuit and an MPS with bond dimension two to iteratively disentangle the target state and construct new layers of the circuit [76]. For a given target state, we can truncate its MPS representation to a bond dimension of two, which yields an MPS that we can exactly write as a single layer of our circuit ansatz. This highly-truncated state should already capture some relevant features of the target state. We can initialize more layers of the

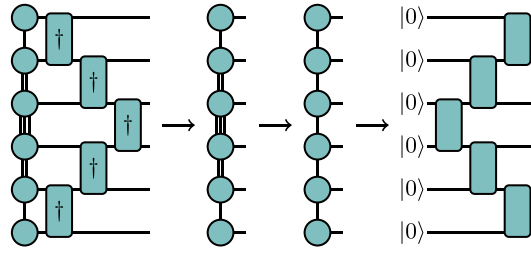


Figure 3. Approximate analytical decomposition of the target state to initialize circuit layers. In the first step, if we already have layers of the circuit, we contract their conjugate transpose with the target state, partially disentangling it. Next, we have the partially disentangled target state and truncate it to an MPS with a bond dimension of two. This truncated MPS corresponds exactly to one layer of a center-sequential circuit, which we can use as a new first layer of the multi-layer circuit.

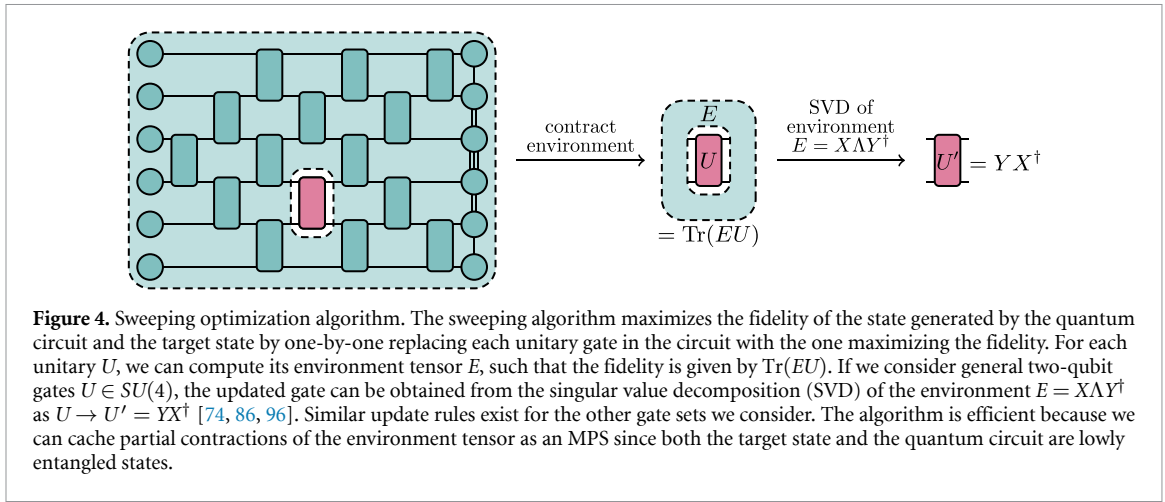
circuit by contracting the conjugate transpose of the already obtained circuit layers with the target state, which gradually disentangles the target state towards the product state $|0\rangle \equiv |0\rangle^{\otimes(n+n_c)}$. The resulting disentangled state can then again be truncated to an MPS with bond dimension two and mapped to one layer of the center-sequential circuit, which is then used as a new first layer in the multi-layer circuit—these steps are graphically shown in figure 3. We can thus add an additional layer to a given pre-existing circuit whose layers do not stem directly from this initialization technique, e.g. one obtained from a previous optimization with one fewer layer.

Note that the initialization technique above works for both the general unitary and special orthogonal two-qubit gates we consider. For general gates in $SU(4)$ we simply drop an irrelevant overall complex phase from the unitary matrices of the truncated, mixed-canonical MPS. For the special orthogonal gates in $SO(4)$, note that the target state is always real so that the resulting matrices from the truncated MPS will always be orthogonal matrices either in $SO(4)$ (as needed) or in $O(4) \setminus SO(4)$ (i.e. real gates with determinant -1 instead of $+1$). In the latter case, since one of the input qubits of the gate will always act on the state $|0\rangle$, we can multiply the gate by a CNOT gate controlled by the qubit in state $|0\rangle$, such that the action of the gate remains unchanged but its determinant changes from -1 to $+1$ as required.

For the sparse gates, we follow the same idea of disentangling the target state, but slightly adapt the procedure. The circuit is made of gates of the form $(V_1 \otimes V_2)\text{CNOT}$, with general single-qubit gates $V_1, V_2 \in SU(2)$, arranged in the center-sequential layout. At the beginning of the circuit, there is an additional initial layer of general single-qubit gates; like this, every CNOT gate in the circuit is effectively surrounded by four single-qubit gates. To add a new layer of the circuit, we initialize the new sparse gates of the layer in the inverse order that they act in the circuit. For each sparse gate, we obtain its two single-qubit-gate components by truncating the MPS of the target state to a product state, and selecting the two single-qubit gates that locally rotate the state $|0\rangle$ into the single-qubit state of the truncated MPS. The new sparse gate is obtained from combining these single-qubit gates with a CNOT gate. Before we move to the next gate, we update the target state by contracting the conjugate transpose of the new sparse gate with it. Once all sparse gates in the new layer have been initialized this way, we truncate the updated target state one final time to a product state, and set the new initial layer of single-qubit gates to be the ones that prepare this product state from the state $|0\rangle$.

Sweeping optimization algorithm. The sweeping algorithm iteratively updates the circuit by sweeping through its gates and replacing one two-qubit gate at a time by the one maximizing the fidelity of the state generated by the quantum circuit and the target state. Each iteration starts at the first layer of the circuit and moves on layer by layer to the final one. Within each layer, we first start from the central gate (the leftmost gate in the $<$ -shape of the layer in figure 4) and update gates by sweeping along the right-canonical arm of the center-sequential circuit, followed by updating the remaining gates in the layer starting from the center sweeping along the left-canonical arm.

To update a given gate U , we compute its environment E by contracting all surrounding tensors such that the fidelity is given by $\text{Tr}(EU)$ —see figure 4. We can then select the gate U —from one of the three gate sets we consider—that maximizes this expression. The contraction of the environment is efficient because we can cache and reuse partial contractions of the tensor network; since the target state, as well as the circuit that is optimized to approximate the target state, are lowly entangled, these partial contractions themselves can always be efficiently stored as an MPS. When we consider general unitary two-qubit gates $U \in SU(4)$, we can obtain the updated gate from the singular value decomposition (SVD) of the environment $E = X\Lambda Y^\dagger$ as



$U \rightarrow U' = YX^\dagger$. This is the unitary matrix guaranteed to maximize $\text{Tr}(EU)$, i.e. the fidelity for a given environment E [86, 96].

If we restrict the two-qubit gates to be special orthogonal $U \in SO(4)$, we must ensure that the updated gate is real and has a unit determinant. The first point is always guaranteed since the target state and all gates in the circuit are real. To ensure the second point, after computing the SVD of the environment as $E = X\Lambda Y^\dagger$, we flip the sign of one of the singular vectors associated with the smallest singular value in Λ , if the new gate does not have the required determinant. This means the update rule for the special orthogonal case—assuming the singular values in Λ are sorted in descending order—is given by $U \rightarrow U' = YCX^\dagger$ with the diagonal matrix $C = \text{diag}(1, 1, 1, \det(YX^\dagger))$, where $\det(YX^\dagger) = \pm 1$ because X and Y are orthogonal matrices. The resulting matrix is the special orthogonal matrix that is guaranteed to maximize the fidelity [97].

If we further restrict to sparse two-qubit gates that are parametrized by a single CNOT gate and some single-qubit gates in $SU(2)$, we can update these single-qubit gates in essentially the same way as the general two-qubit gates in $SU(4)$ before. The only difference is that the matrix dimensions of the contracted environment tensors are smaller.

In practice, before moving on to the next steps of the algorithm, we iterate the two steps above to grow the number of layers to the desired number [77]. In each iteration, we initialize an additional layer according to the analytical decomposition and optimize all layers for some iterations with the sweeping algorithm.

Two-qubit gate decomposition. In the preceding steps, we performed the optimization using (i) general two-qubit gates in $SU(4)$, (ii) special orthogonal two-qubit gates in $SO(4)$ and (iii) sparse two-qubit gates using only a single CNOT gate. Until now, though, we have stored them as 4×4 matrices, so we need to decompose them into hardware-compatible CNOTs and parametrized single-qubit gates.

General two-qubit gates in $SU(4)$ can be decomposed into three CNOTs and several single-qubit gates [84]. The first and last gates in this decomposition are single-qubit gates, so after decomposing every two-qubit gate in the circuit, there are consecutively appearing single-qubit gates that can be merged into a single one. After this merging of single-qubit gates, we have one layer of single-qubit gates acting on the initial state $|0\rangle$, and the remaining two-qubit gates without the redundant single-qubit gates are then parametrized as

$$U = \begin{array}{c} \oplus \\ \text{---} R_z \text{---} \\ \oplus \\ \text{---} R_y \text{---} \end{array} \begin{array}{c} \oplus \\ \text{---} R_z \text{---} \\ \oplus \\ \text{---} R_y \text{---} \end{array} \begin{array}{c} \oplus \\ \text{---} V_1 \text{---} \\ \oplus \\ \text{---} V_2 \text{---} \end{array}, \quad (4)$$

where R_α denote single-qubit rotations about the axis $\alpha \in \{x, y, z\}$ and $V_i = R_z(\phi_i)R_y(\theta_i)R_z(\lambda_i)$ are general single-qubit gates in $SU(2)$. Thus, there are nine free parameters per two-qubit gate corresponding to the rotation angles. Together with the parameters of the initial layer of single-qubit gates, this gives a total of $9(L-1)d + 2L$ parameters for a center-sequential circuit with L qubits and d layers.

Two-qubit gates from the special orthogonal group $SO(4)$ can be decomposed using two CNOTs and six single-qubit y -rotation gates [85]. After merging consecutive single-qubit gates, we again have an initial layer

of single-qubit gates acting on the initial state $|0\rangle$, and two-qubit gates parametrized as

$$U = \begin{array}{c} \oplus \\ \text{---} R_y \text{---} \oplus \\ | \\ \text{---} R_y \text{---} \\ \oplus \end{array} \quad (5)$$

which now only require four R_y gates in addition to the two CNOT gates. In total, a center-sequential circuit with L qubits and d layers using special orthogonal two-qubit gates thus has $4(L-1)d + L$ free parameters.

Finally, the sparse gates, by definition, consist of only a single CNOT gate and single-qubit gates. We again have an initial layer of single-qubit gates acting on the initial state $|0\rangle$ and each two-qubit gate is then parametrized as

$$U = \begin{array}{c} \text{---} R_y \text{---} \\ | \\ \oplus \text{---} R_y \text{---} \end{array} \quad (6)$$

Note that in our case, we can always choose the general single-qubit gates to be R_y gates, since the single-qubit gates obtained from the sweeping algorithm are real and we can modify them to ensure a positive determinant. For this, we focus on one qubit at a time, starting from the uppermost qubit (see figure 2) and then moving down one by one, and for each qubit separately we consider the set of single-qubit gates acting on it. We start with the last single-qubit gate acting on the given qubit, update the gate if necessary, and afterwards successively update the preceding single-qubit gate if necessary. To check whether we need to update a single-qubit gate, we can check its determinant: If it is positive, the single-qubit gate is an R_y gate and we can move on to the next gate. If its determinant is negative, we need to modify the gates in the circuit. We decompose the gate as $R_y X$ and move the Pauli- X gate past the preceding CNOT gate. If it needs to move past the control of the CNOT gate, we use the identity $(X \otimes \mathbb{1})\text{CNOT} = \text{CNOT}(X \otimes X)$ which moves the Pauli- X gate past the CNOT gate and creates another Pauli- X gate on the lower neighboring qubit. The single-qubit gates in the circuit can be updated by absorbing these Pauli- X gates, and then we move on to the next single-qubit gate (i.e. the upper one of the two that were just updated). In the other case, where the Pauli- X gate needs to move past the target of the CNOT gate, the gates commute and we can simply update the preceding single-qubit gate by absorbing the Pauli- X gate; then, we move on to the next single-qubit gate (i.e. the one that was just updated). Once we arrive at the first single-qubit gate acting on the qubit, i.e. the one that acts on the state $|0\rangle$, we can eliminate a potentially needed Pauli- X gate by making use of the relation $X|0\rangle = |1\rangle = R_y(\pi)|0\rangle$ and merging the y -rotation gates. Once we reach the last (bottommost in figure 2) qubit, note that only the target of CNOT gates acts on this qubit, so any Pauli- X gates can be commuted past the CNOT gates and no new Pauli- X gates are created by this commutation. Thus, the algorithm terminates after these final substitutions and afterwards the circuit consists of only R_y and CNOT gates. The center-sequential circuit using sparse gates with L qubits and d layers therefore has $2(L-1)d + L$ free parameters in total.

BFGS optimization. Far from the optimal circuit, the sweeping algorithm substantially improves the fidelity with the target state with each update. However, close to convergence, the sweeping algorithm performs only small update steps in the direction of a maximum, and thus acts similarly to a simple gradient ascent optimizer [88]. Gradient ascent only makes use of information about the gradient of the function (making it a first-order method) leading to a linear convergence rate. Moreover, we only update a single tensor at a time, which can be seen as a variant of coordinate descent. This leads to very slow convergence close to a maximum of the fidelity. To speed up convergence, we therefore switch to the BFGS algorithm [87] to optimize the parameters in the circuit after the gate decomposition. The BFGS algorithm is also an iterative method, similar to gradient ascent, except that the update rule additionally incorporates information about the Hessian of the function, making it a second-order method. It belongs to the class of quasi-Newton methods, since it approximates Newton's method by estimating the inverse Hessian based on gradient evaluations. Theoretically, Newton's method promises a faster, quadratic convergence. Moreover, at each step, we update all parameters at once allowing to traverse the cost landscape more efficiently than the coordinate descent of the sweeping algorithm. Combined, this leads to a faster convergence and higher fidelity after the optimization compared to only using the sweeping algorithm.

Note that, in principle, we can use the same methods as for the sweeping algorithm to compute the gradient of each gate in the circuit by contracting its environment and caching partial contractions of the circuit as MPSs. This approach formally scales only polynomially with the system size and the number of parameters. However, current state-of-the-art implementations of statevector simulations are so highly optimized that—even though they formally scale exponentially with the number of qubits—for the system

sizes we consider here (up to 19 qubits) they still outperform our Python implementation of the tensor-network-based gradient computation. Thus, for the results presented in the following section, we use statevector simulators and automatic differentiation for the gradient computation for the BFGS optimization.

3.3. Efficient circuit representations of typical machine learning datasets

We apply the algorithm described in section 3.2 to find circuits which encode four standard machine learning datasets: the MNIST [48, 49], Fashion-MNIST [50], CIFAR-10 [51], and Imagenette [52] datasets. We will use the same datasets to show the results of quantum classifiers in section 4. For the implementation of the algorithm, we use Ray [98] to parallelize the execution of the circuit optimization tasks. For the gradient-based optimization, we use PennyLane [53] in combination with the just-in-time compilation and vectorization capabilities of JAX [99], and SciPy's BFGS implementation [100].

The MNIST dataset [48, 49] is a simple and widely used dataset for training machine learning models. It contains grayscale images of handwritten digits between '0' and '9', and associated labels indicating the correct digit. The original images have 28×28 pixels. Here, we use bilinear interpolation to resize them to 32×32 pixels making them suitable for processing on a quantum computer. The class distribution over the 70 000 images is approximately uniform, with each class representing between 9% and 11% of the dataset. Current state-of-the-art classical CNNs achieve a validation accuracy of above 99.7% on this dataset [101–104].

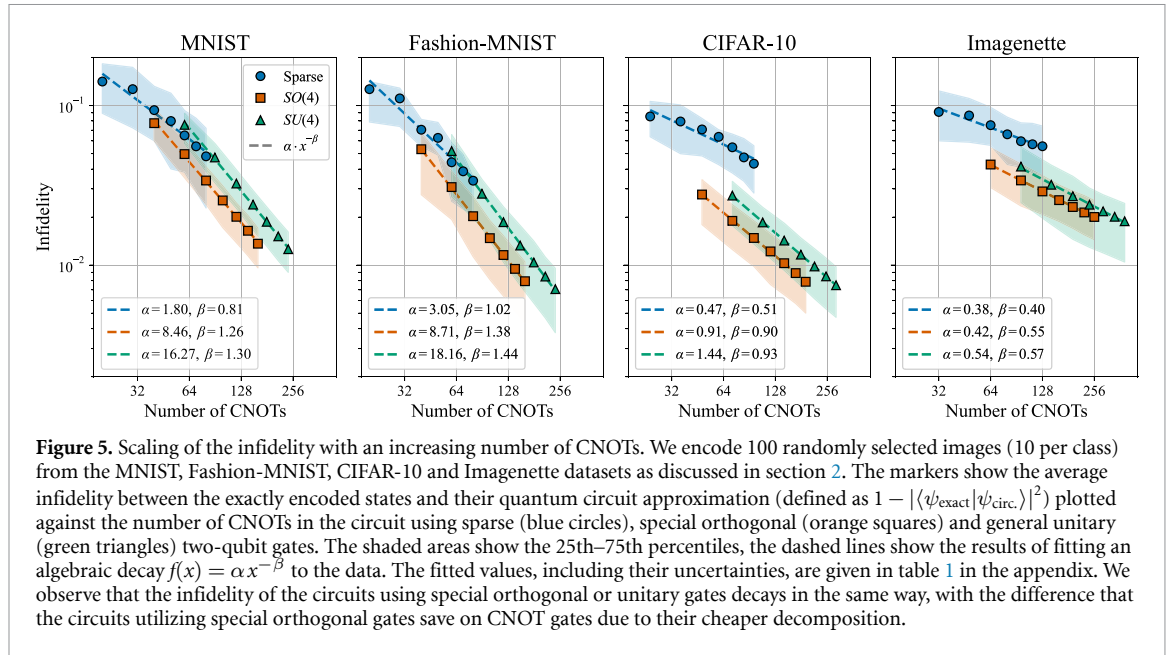
The Fashion-MNIST dataset [50] was introduced as a more challenging alternative to MNIST, after it became apparent that MNIST was too easily solved and no longer posed a significant challenge for more sophisticated classification models. The dataset also features 70 000 grayscale images with an original resolution of 28×28 pixels, which we again resize to 32×32 pixels using bilinear interpolation. Instead of handwritten digits, the images feature different clothing articles and the associated labels denote the ten different classes of fashion articles, e.g. T-shirts, trousers or sneakers. The images are equally distributed over the ten classes. While Fashion-MNIST is a more complex dataset than MNIST, current classical state-of-the-art neural networks nonetheless achieve around 94%–98% validation accuracy [104–106], making it still a relatively simple toy dataset.

The CIFAR-10 dataset [51] is a more realistic dataset, and also widely used as a benchmark in machine learning and computer vision. It consists of 60 000 32×32 -pixel color images. The images are photographs of objects belonging to ten different classes, such as airplanes, automobiles or birds, with an equal number of images per class. Being a more challenging dataset, classical state-of-the-art models 'only' achieve a validation accuracy of 92%–95% [103, 104, 107].

Finally, we consider the Imagenette dataset [52], which is a subset of the more well-known and larger ImageNet dataset [108]. It includes color images of ten more easily differentiated classes from the ImageNet dataset, e.g. cassette players, garbage trucks and parachutes, and is intended as a smaller and simpler stand-in for the huge ImageNet dataset. It comprises only 13 394 images. The resolutions and aspect ratios vary between images, so to work with a uniform resolution we take the largest square section of each image and resize them to 128×128 pixels using bilinear interpolation. Classical neural networks achieve around 94% validation accuracy on this dataset [52].

We test the optimization algorithm described in the previous subsection on images from these datasets. First, we want to investigate how the approximation quality of the optimized circuit improves when increasing the number of layers in the circuit. For this, we consider the infidelity of the circuit with its target state, defined as $1 - |\langle \psi_{\text{exact}} | \psi_{\text{circ.}} \rangle|^2$. To compare the circuits using different two-qubit-gate primitives, i.e. general unitary gates, special orthogonal gates or sparse gates, we compare them by the total number of CNOTs used in the circuit. This is because CNOT gates typically incur larger errors than single-qubit gates in many hardware realizations, so for a faithful implementation on a quantum computer we want to minimize the total number of CNOT gates. We focus on CNOT gates as a metric because we intend to offer circuits that can be executed on today's noisy hardware, as has been demonstrated for similar circuit architectures in references [45, 46]. A complementary view would be to see this state preparation as a subroutine on a future fault-tolerant quantum computer, where (when working with a Clifford+ T gate set) implementing Clifford gates is free but the implementation of T gates becomes costly. We show a similar comparison with the number of T gates in appendix B.

As a representative example for this comparison, we take 100 images from each of the four datasets, 10 random images from each class, and optimize circuits with 2–8 layers for each of the three different two-qubit gates. The results are shown in figure 5, where blue circles show the average infidelity for circuits using sparse two-qubit gates, orange squares the average infidelity for special orthogonal two-qubit gates and green triangles the average infidelity for general unitary two-qubit gates. The shaded areas correspond to the 25th–75th percentiles. The dashed lines show algebraic fits to the data, the obtained values including uncertainties are summarized in table 1 in the appendix.



We can see for all datasets that the infidelity decays algebraically with the number of CNOTs, which is proportional to the number of layers of the circuit. This is not unexpected, as these circuits are similar to MPSs with the number of layers playing a role akin to the bond dimension, and for MPSs such an algebraically decaying infidelity can be derived [44]. For the circuits using the sparse gates, we see that while they use the fewest CNOTs per layer, they also make use of them in the least efficient way, displaying the largest infidelity for a fixed number of CNOT gates out of all three gate types. The infidelities of the circuits using general unitary or special orthogonal two-qubit gates decay at the same rate. Since the target states are real, it seems reasonable to expect that a circuit using real gates can approximate the target states just as well as a circuit using complex-valued gates with the same number of layers. From this intuition, one would expect the ratio of prefactors of the algebraic decay to roughly match the ratio $2/3 \approx 0.67$ of CNOTs required for implementing a special orthogonal or general unitary gate. For the low-resolution datasets with 32×32 pixels, the empirical ratios are even smaller with $8.46/16.27 \approx 0.52$ for MNIST, $8.71/18.16 \approx 0.48$ for Fashion-MNIST, and $0.91/1.44 \approx 0.63$ for CIFAR-10, indicating that the circuits with special orthogonal gates are even more efficient than expected on these datasets. However, on the higher resolution dataset Imagenette with 128×128 pixels, the ratio of prefactors is $0.42/0.54 \approx 0.78$ which is larger than the expected ratio. Still, across all four datasets, the special orthogonal gates clearly perform best out of the three considered gate variants, so in the following we will focus on this gate set.

Next, we want to examine the scaling of the infidelity with the image resolution for a fixed number of layers. For this, we consider 100 randomly selected images from the Imagenette dataset which originally have a resolution higher than 256×256 pixels, so that we can reasonably tune the image resolution. This is in contrast to the previously used images from the MNIST, Fashion-MNIST and CIFAR-10 datasets, which at 32×32 pixels already have a rather low resolution. Figure 6 shows the resulting scaling of the average infidelity when changing the $2^{n/2} \times 2^{n/2}$ -pixel image resolution through $\frac{n}{2} \in \{2, 3, \dots, 8\}$. Different colors and marker shapes show results for different numbers of layers, the shaded regions show the 25th–75th percentiles. The black stars show the results for optimizing circuits with four layers to approximate states with amplitudes sampled from a normal distribution before normalization, i.e. real Haar-random states. While such random states cannot be effectively described by low-depth circuits, the states encoding image data are well-captured with far smaller infidelities. Generally, we see the trend that more circuit layers help capture the state more accurately (as already seen in figure 5), while a higher resolution introduces a greater complexity into the image making the approximation more difficult.

As we change the image resolution, we expect two distinct behaviors. For a high image resolution, a further increase of the resolution does not introduce a lot of new features to the image, but only sharpens the already visible details. As such, the state should not be much harder to represent than at a lower resolution. From the MPS representation, where a truncated Fourier series of the image can be represented exactly, we expect that for a high resolution there are only finite-size effects that are exponentially small in the number of qubits [44]. For very low image resolutions, changing the resolution can still drastically change the image and introduce a lot of new information which has to be captured by the approximation. Thus, at low image

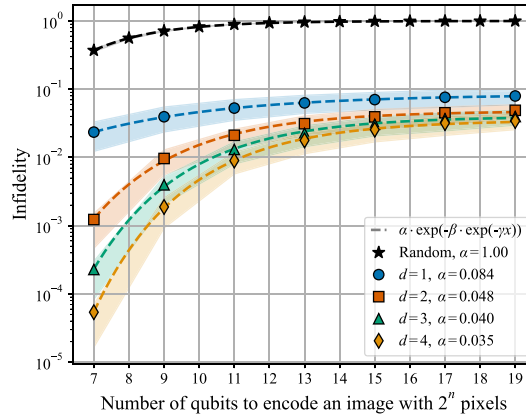


Figure 6. Scaling of the infidelity with increasing image resolution. We consider 100 randomly selected color images (with an original resolution of at least 256×256) from the Imagenette dataset and rescale them using bilinear interpolation to different resolutions ranging from 4×4 to 256×256 pixels. The plot shows the average infidelity between the exactly encoded states and their quantum circuit approximation (using special orthogonal two-qubit gates) as a function of the number of qubits, which is directly related to the image resolution. Different marker shapes and colors denote circuits with an increasing number of layers d , the shaded areas show the 25th–75th percentiles. The black stars show results for approximating real Haar-random states with quantum circuits using special orthogonal two-qubit gates with $d = 4$ layers. While for the random states the infidelity quickly saturates at one, indicating that they cannot be effectively approximated by a low-depth circuit, the infidelity of the image states seems to saturate for a fixed number of layers as the image resolution increases, suggesting that the circuit approximation works even for high-resolution images. The dashed lines act as a guide to the eye, they correspond to a fit of a Gompertz function (see equation (7)) to the data, where α captures the saturation value of the infidelity. The exact values of all fit parameters are given in table 2 in the appendix.

resolutions we expect the generic behavior where the infidelity of two distinct quantum states grows exponentially with the number of qubits. However, the actual value of the infidelity must be very small in this regime, as a circuit with sufficiently many layers can represent the target state exactly for sufficiently few qubits (e.g. for two qubits a single-layer circuit can prepare the state exactly). At some intermediate value of the infidelity, there has to be a crossover between the two regimes. A function that captures such a crossover is the Gompertz function, defined as

$$f(x) = \alpha \exp(-\beta \exp(-\gamma x)). \quad (7)$$

For small inputs $\gamma x \ll 1$, this becomes an exponential increase: $\alpha \exp(-\beta(1 - \gamma x)) \propto \exp(\beta\gamma x)$. For large inputs $\gamma x \gg 1$ this converges to the value of the prefactor α up to exponentially small corrections, as $\alpha(1 - \beta \exp(-\gamma x)) = \alpha + \mathcal{O}(\exp(-\gamma x))$. The dashed lines in figure 6 show a fit of the Gompertz function to the data. All values of the fit parameters and their uncertainties are given in table 2 in the appendix. We see that it phenomenologically captures the behavior in all cases. Importantly, it indicates that the average infidelity of the states seems to saturate to some value smaller than one as the image resolution is increased. This suggests that the number of layers in the circuit does not need to grow asymptotically with the system size to keep the infidelity below a certain error threshold, and thus means that the state preparation of MCRQI-encoded images with fixed error only scales linearly with the number of qubits.

Making use of the success that the circuits employing special orthogonal gates have shown in these two settings, we have optimized circuits using them for all images in the MNIST [48, 49], Fashion-MNIST [50], CIFAR-10 [51], and Imagenette [52] datasets. We provide the optimized circuits as PennyLane datasets at <https://pennylane.ai/datasets/collection/low-depth-image-circuits>.

4. Benchmarking quantum classifiers

After bringing the classical data from the MNIST [48, 49], Fashion-MNIST [50], CIFAR-10 [51] and Imagenette [52] datasets into a form in which we can efficiently load them on a quantum computer, we use them to evaluate the performance of different classifiers. While finding a quantum classifier that is competitive with classical methods is beyond the scope of this work, these benchmarks demonstrate the utility of the datasets while showing some shortcomings of current quantum classifiers and highlighting potentially fruitful directions to explore. The tested classifiers include VQCs—both standard and a nonlinear variant—as well as SVMs with kernels induced by the data encoding. When we use the patched or multi-copy encoding, the number of qubits grows, and therefore the simulation cost increases, so we switch from circuit-based to tensor-network-based classifiers. We also compare against a classical CNN representing

a state-of-the-art classical method. To ensure a robust evaluation, we split the data into training and validation sets using five-fold cross-validation. After evaluating the models on the uncompressed datasets, we assess the performance of the classifiers on the compressed datasets.

For the training of the models in the following, we use Ray [98] and GNU Parallel [109] to parallelize multiple model instances. For state vector simulations, we use PennyLane [53] in combination with the just-in-time compilation and vectorization capabilities of JAX [99]. The tensor-network simulations are likewise implemented directly in JAX, leveraging the same performance-optimizing features.

4.1. Quantum classifiers

In the following, we introduce the quantum classifiers considered in this work. First, we discuss the *explicit* approach to learning classification tasks on a quantum computer, where a VQC is trained as a quantum circuit classifier to construct a decision boundary for the classification problem. For this setup, we also propose a way to introduce *nonlinearities* into each layer of the VQC processing the input data, similar to how classical neural networks process data nonlinearly at each step. Then, we consider kernel-based classifiers, which can be seen as an *implicit* implementation of the linear quantum circuit classifier.

Quantum circuit classifiers. For a supervised classification problem like we consider here, the goal is to learn a decision function $f_\ell(\mathbf{x}; \boldsymbol{\theta})$ parametrized by some variational parameters $\boldsymbol{\theta}$ that, given an input $\mathbf{x}$, outputs a number for each possible class label ℓ . The largest number of these outputs should correspond to the correct class. The model is trained by tuning the parameters $\boldsymbol{\theta}$ such that they minimize some loss function, which quantifies the error the model is making on some training set, usually via gradient descent methods. The typical approach in variational quantum computing is to express this learnable decision function $f_\ell(\mathbf{x}; \boldsymbol{\theta})$ as the expectation value of a quantum circuit: starting from a state that encodes a data point $|\psi(\mathbf{x})\rangle$, we apply some parametrized quantum circuit $V(\boldsymbol{\theta})$ and, finally, measure an observable O_ℓ for each class [20, 110, 111]. This way, the decision function is

$$\begin{aligned} f_\ell(\mathbf{x}) &= \langle \psi(\mathbf{x}) | V(\boldsymbol{\theta})^\dagger O_\ell V(\boldsymbol{\theta}) | \psi(\mathbf{x}) \rangle \\ &= \text{Tr}[O_\ell(\boldsymbol{\theta}) \rho(\mathbf{x})], \end{aligned} \quad (8)$$

where we have collected the input state into a density matrix $\rho(\mathbf{x}) \equiv |\psi(\mathbf{x})\rangle\langle\psi(\mathbf{x})|$ and reinterpreted the variational circuit as a basis transformation of the measured observable $O_\ell(\boldsymbol{\theta}) \equiv V(\boldsymbol{\theta})^\dagger O_\ell V(\boldsymbol{\theta})$. From this rewriting, we can directly see that the quantum model will be a linear function of the input density matrix. Each observable $O_\ell(\boldsymbol{\theta})$ draws a hyperplane in the space of Hermitian matrices separating data points corresponding to label ℓ from data points corresponding to other classes [18, 19, 69]. So, for the classifier to capture sufficiently complex relations it is expected that a more nonlinear mapping from the input data $\mathbf{x}$ to the quantum state $\rho(\mathbf{x})$, like the patched or multi-copy encodings discussed in section 2, is needed.

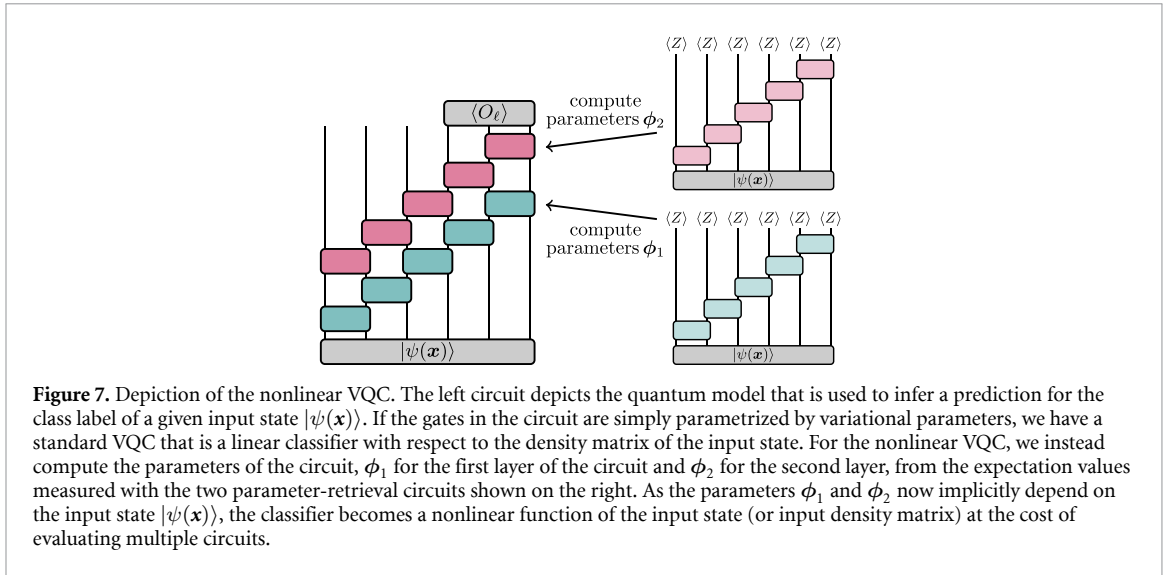
In our case, the input states are given by the states discussed in the preceding sections. The circuit architecture of the VQC is a sequential circuit (see figure 7, on the left without the additional circuits on the right) with general two-qubit gates in $SU(4)$ [43, 46]. We parametrize the gate by explicitly expressing the generator as a linear combination of the fifteen non-identity Pauli strings. When the system has L qubits, there are thus $15(L-1)$ trainable parameters per layer. Note that, if the input states are also given as sequential circuits, this circuit architecture avoids barren plateaus for a number of layers up to logarithmic in the number of qubits [112–114]. For the measured observables, we take a weighted sum of the probabilities of all sixteen bitstrings in the computational basis on the last four qubits. Specifically, the observable is given by

$$O_\ell = \sum_{j=0}^{15} A_{\ell j} |j\rangle\langle j| + b_\ell \mathbb{1}, \quad (9)$$

with the 10×16 -matrix of weights A and the 10-dimensional bias vector $\mathbf{b}$, that are also optimized during training [46].

For training, we feed the output of the model into a softmax function with a temperature of $T = 1/128$ before using the categorical cross-entropy loss. (This combination is sometimes also referred to as the softmax loss function.) We train for 100 epochs using the Adam optimizer [115] with a learning rate of $8 \cdot 10^{-4}$ and a batch size of 100. Both the variational parameters of the quantum circuit and the parameters of the observable are initialized uniformly at random in the range $[0, 1]$.

Nonlinear quantum circuit classifiers. To address the limitations of linear models, the framework of data reuploading has been proposed [11–13]. Repeating data-dependent gates several times in the circuit can make the resulting state depend very nonlinearly on the input data. The data-dependent operations can be



repeated in space, for example like the multi-copy encoding discussed in section 2 (which we will consider in the following subsection), or repeated in time, when the same gate appears several times at different depths of the circuit. The latter strategy allows to show that even a single qubit can approximate any continuous function [11], rendering it a universal classifier in principle, analogous to the universal approximation theorem for neural networks [116]. However, this specific strategy is not applicable to the encodings considered in this work, since our circuits do not implement a general basis transformation. Instead, they only realize an isometry that maps the initial state $|0\rangle$ to the quantum state approximately representing the image.

Here, we explore an alternative route to add nonlinearity to the classifier, and propose a quantum circuit model that nonlinearly processes the input density matrix by making the circuit's gates explicitly dependent on the input state. A schematic of the model is depicted in figure 7. The nonlinear quantum circuit classifier comprises two main components: a parametrized quantum circuit that acts as the classifier and parameter-retrieval circuits that are used to compute the parameters of the gates in the classifier.

For the classifier quantum circuit, we use the same setup as for the usual, linear VQC classifier. That is, we use a sequential circuit (see figure 7, left) with several layers, and each gate a general two-qubit gate in $SU(4)$. For a system with L qubits, this means $15(L-1)$ parameters per layer. The observable is also the same trainable weighted sum of probabilities of the computational-basis bitstrings of the final four qubits as for the linear model—see equation (9).

The distinctive feature of this model is that the gates in the classifier quantum circuit do not have fixed parameter values, but rather the parameters are calculated for each new input state $|\psi(\mathbf{x})\rangle$ via the parameter-retrieval circuits. For each layer m of the main classifier circuit, there is a separate parameter-retrieval circuit $V_m(\theta_m)$ (with variational parameters θ_m). To compute the parameters for the m th layer of the main classifier circuit, we first compute the vector of single-qubit Pauli-Z expectation values σ_m with respect to the state obtained by applying the parameter-retrieval circuit to the input state, i.e.

$$\sigma_{m,j} = \langle \psi(\mathbf{x}) | V_m(\theta_m)^\dagger Z_j V_m(\theta_m) | \psi(\mathbf{x}) \rangle. \quad (10)$$

For a system with L qubits, this gives only L numbers, while we need $15(L-1)$ parameters for the classifier circuit. To match the dimensions, we multiply the vector of expectation values by a $15(L-1) \times L$ -dimensional weight matrix W_m . The final parameters for the classifier circuit are then obtained as

$$\phi_m = \pi \tanh(W_m \sigma_m), \quad (11)$$

where the factor of π and the hyperbolic tangent are applied element-wise to rescale the gate parameters between $-\pi$ and π . We keep the weight matrix W_m as additional variational parameters of the model. The parameters ϕ_m , which implicitly depend on the input state $|\psi(\mathbf{x})\rangle$, are then used as the gate parameters of the m th layer of the main classifier circuit. Importantly, the dependence of the parameters ϕ_m on the input state makes the resulting classifier circuit a nonlinear function of the input density matrix—this advantage, however, comes at the cost of having to evaluate an increased number of circuits on the quantum computer.

For the optimization of the model, we use the same setup and hyperparameters as for the training of the linear quantum circuit classifier.

Quantum kernel methods. Instead of constructing a quantum circuit that explicitly computes the label of an encoded data point, we can also implicitly use a quantum computer to infer the label, by first using a quantum computer to find inner products of a new data point with a reference set of labeled data points and then classically obtaining the label from weighting the inner products [18, 19, 117]. This is also the idea of kernel methods [118], the inner product leading to the specific kernel function $\kappa(\mathbf{x}, \mathbf{y}) = \text{Tr}[\rho(\mathbf{x})\rho(\mathbf{y})]$ [18, 19, 69]. The classifier is then built as a linear combination of kernel functions, where one of the inputs is fixed to elements $\mathbf{x}_n$ of the training set, as

$$f_{\ell}(\mathbf{x}; \boldsymbol{\alpha}) = \sum_n \alpha_{\ell n} \kappa(\mathbf{x}_n, \mathbf{x}). \quad (12)$$

If the training set is unrestricted and comprises all possible input data, this recovers the same set of linear classifiers as the linear quantum circuits discussed before [69, 117]. However, even for a restricted training set, the representer theorem guarantees that a model of the form above achieves minimal loss when evaluated on the same training set [118], and there are known methods for finding the optimal parameters of such a model. Still, while the training loss may be optimal, no such statement can be made about the loss on unseen samples like the validation set, and kernel methods tend to overfit quite strongly in practice [117].

With the data encodings we use, the resulting quantum states remain lowly entangled and can be represented as MPSS. Therefore, the inner products can be computed efficiently classically and this computation does not require a quantum computer. Nonetheless, the kernel-based classifier is closely related to the explicit quantum circuit classifiers, and serves as an interesting model for comparison. As a specific kernel-based model, we consider a SVM. We use the `svm.SVC` implementation of scikit-learn [119], which implements a polynomial kernel

$$\kappa(\mathbf{x}, \mathbf{y}) = (\gamma \cdot \langle \psi(\mathbf{x}) | \psi(\mathbf{y}) \rangle + C)^d \quad (13)$$

where we set the kernel prefactor $\gamma = 1$, the regularization parameter $C = 0$ and the degree $d = 2$ to reproduce the quantum kernel. (Other parameters of the scikit-learn implementation are kept at the default values.) Multi-class classification is handled via a one-vs-rest strategy.

Performance of the quantum classifiers. Figure 8 compares the classification accuracies of the linear VQC (orange), the nonlinear VQC (blue) and the SVM with quantum kernel (turquoise) on the three benchmark datasets MNIST, Fashion-MNIST and CIFAR-10. Training the circuit classifiers on the complete Imagenette dataset is not feasible because of the larger number of qubits needed. The solid lines indicate the accuracy on the training set, while dashed lines show the accuracy on the validation set, and shaded areas depict one standard deviation from five-fold cross-validation.

In addition to training the quantum models, we also trained a linear neural network (i.e. a matrix transformation) to identify how easily the datasets can be classified linearly. We use the same resolution of the images as for the quantum datasets and standardize them to zero mean and unit variance. As for the quantum classifiers, to compute the mean and the standard deviation we employ five-fold cross-validation. We train the model for 100 epochs using the Adam optimizer with default parameters, and, in the following discussion, report the training and validation accuracy of the epoch with the highest validation accuracy.

On the MNIST dataset (left panel), all classifiers achieve a high accuracy above 90% for sufficiently many layers. The nonlinear VQC reaches the best accuracy, which saturates around 95% validation accuracy for moderately deep circuits (4–8 layers). With this, it outperforms both linear classifiers, the linear VQC, which saturates a few percentage points lower at 92%, and also narrowly the SVM, which achieves an accuracy of $94.2 \pm 0.2\%$. The linear neural network achieves an accuracy of $93.5 \pm 0.2\%$ on the training and $92.3 \pm 0.2\%$ on the validation set, which is similar to accuracy of the linear quantum models.

The increased complexity of the Fashion-MNIST dataset (central panel) leads to all models achieving lower accuracies than on the MNIST dataset, but overall the performance is still promising. When using only a few layers, the nonlinear VQC still outperforms its linear variant, however, the validation accuracies of both classifiers saturate at a similar value, and only the training accuracy of the nonlinear VQC remains about 3% higher at larger depths. Both classifiers are beaten by the SVM, which achieves the highest accuracy of $86.3 \pm 0.3\%$. With $86.7 \pm 0.5\%$ on the training and $85.6 \pm 0.4\%$ on the validation set, the linear neural network closely matches the performance of the linear quantum variants.

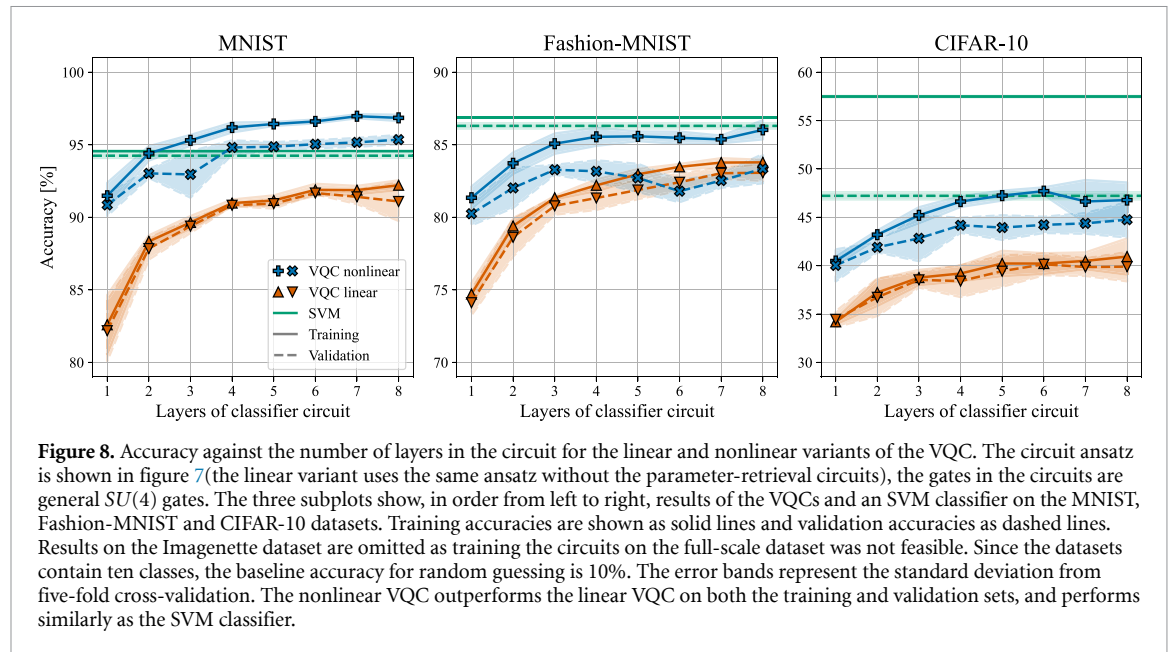


Figure 8. Accuracy against the number of layers in the circuit for the linear and nonlinear variants of the VQC. The circuit ansatz is shown in figure 7 (the linear variant uses the same ansatz without the parameter-retrieval circuits), the gates in the circuits are general $SU(4)$ gates. The three subplots show, in order from left to right, results of the VQCs and an SVM classifier on the MNIST, Fashion-MNIST and CIFAR-10 datasets. Training accuracies are shown as solid lines and validation accuracies as dashed lines. Results on the Imagenette dataset are omitted as training the circuits on the full-scale dataset was not feasible. Since the datasets contain ten classes, the baseline accuracy for random guessing is 10%. The error bands represent the standard deviation from five-fold cross-validation. The nonlinear VQC outperforms the linear VQC on both the training and validation sets, and performs similarly as the SVM classifier.

The performance of all classifiers decreases significantly on the more realistic CIFAR-10 dataset (right panel), with validation accuracies below 48% for all classifiers. While this is still notably better than the baseline of 10% for random guessing, it is also much worse than the performance on the previous two datasets. We again observe the trend that the nonlinear VQC outperforms the linear VQC, saturating at almost 45% validation accuracy compared to around 40% for the linear model. The nonlinear VQC, however, does not beat the SVM classifier, which achieves a slightly higher validation accuracy of $47.2 \pm 0.5\%$; the SVM classifier, though, also overfits strongly with a testing accuracy of $57.5 \pm 0.2\%$. The linear neural network baseline attains $43.76 \pm 0.5\%$ on the training and $40.4 \pm 0.5\%$ on the validation set, comparable to the linear quantum models.

Overall, we observe that the nonlinear VQC tends to outperform the linear VQC on all three datasets, suggesting that the added expressivity of allowing the circuit parameters to depend on the input state leads to a higher classification accuracy. Note that while the nonlinear VQC also has significantly more variational parameters for the same number of layers, the difference in performance does indeed seem to stem from the added nonlinearity. The accuracy of the linear VQC saturates for a large number of layers, indicating that adding more layers until the number of parameters in the two models matches will not improve performance. This saturation value, however, is distinctly smaller than the accuracy of the nonlinear VQC. While the improved accuracy of the nonlinear VQC over the linear VQC makes it a promising model, the added nonlinearity still does not make it reliably beat the SVM classifier (even though their performance is very close). Smarter parameter initialization schemes or more structured methods for computing the circuit parameters from the expectation values of the parameter-retrieval circuits could potentially push the model beyond the accuracy of the SVM classifier.

From these results, we also see that it is important to test the quantum classifiers on a range of different datasets. Had we considered only the MNIST dataset, we might have concluded that the nonlinear VQC already outperforms the SVM classifier, while the other two datasets show that this is not reliably the case. On the other hand, had we only tested on the Fashion-MNIST dataset we might have assumed that the nonlinear VQC shows no improvement over the linear VQC, since both converge to the same validation accuracy on this dataset with an increased number of layers. More generally, we see that the classifiers perform well on the simple datasets, especially the MNIST and to some extent also on the Fashion-MNIST dataset, but this does not translate to realistic datasets like CIFAR-10, where the performance drops off significantly.

4.2. Tensor-network classifiers

The FRQI- and MCRQI-based encodings of the data offer a resource-efficient mapping from the data into the Hilbert space, using only slightly more qubits than are needed for the Hilbert space dimension to match the data dimension. However, this mapping might be too efficient, in the sense that it offers too little freedom to subsequently allow linear quantum classifiers to draw a hyperplane that is able to separate the different classes. By increasing the number of qubits used in the encoding, the data is embedded in a higher-dimensional space and it becomes easier to find a hyperplane that adequately separates the data. In

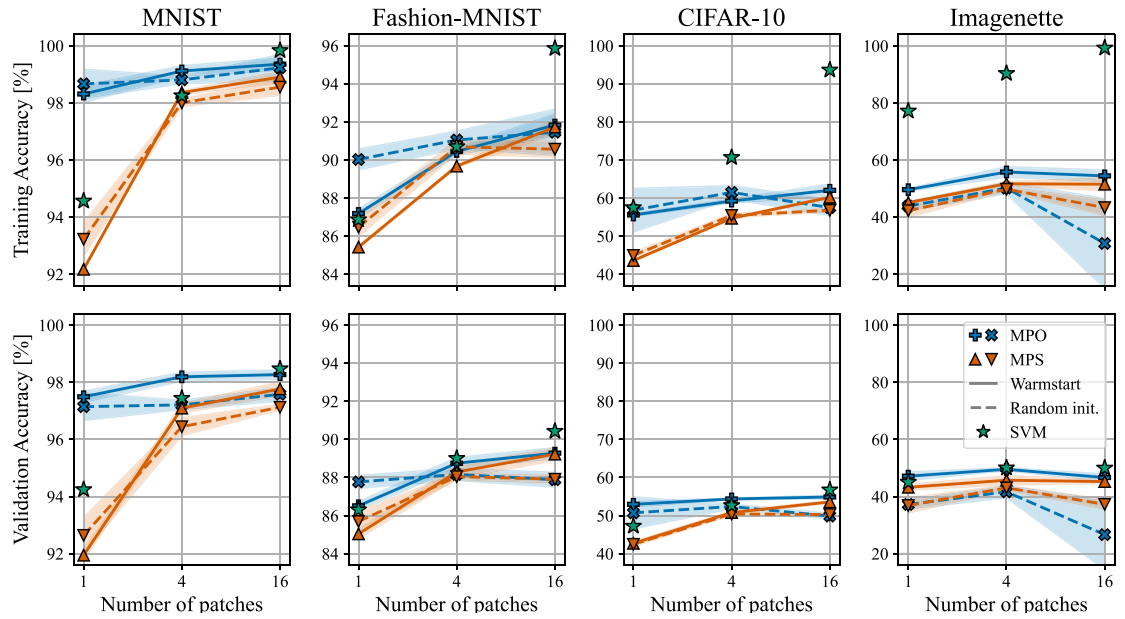


Figure 10. Accuracy of the tensor-network and SVM classifiers versus the number of patches used in the encoding. Both the training (top row) and validation accuracy (bottom row) are shown, left to right, on the four datasets MNIST, Fashion-MNIST, CIFAR-10 and Imagenette. Results for the MPS and MPO classifiers are shown in orange and blue, for both the warm start initialization as solid lines and random initialization as dashed lines. The results for the SVM are shown as green stars. Since the datasets contain ten classes, the baseline accuracy for random guessing is 10%. The error bands represent the standard deviation from five-fold cross-validation.

combinations of quadratic functions of the input state compared to only linear functions of the input state for the latter.

For the MPO classifier, we consider the same initialization techniques as for the MPS classifier, i.e. random initialization and pretraining. For the random initialization, the entries of the MPO tensors are drawn from the same normal distribution with zero mean and standard deviation $\sigma = 1/\sqrt{\chi}$ as the MPS tensors. For the initialization via pretraining, the only change is that we now sum the outer products of the input states instead of the states directly, so the output is an MPO instead of an MPS, and we afterwards normalize the MPO to have Frobenius norm $\sqrt{2^L} 10$. Also for the MPO classifier we set the maximal bond dimension to $\chi = 32$.

For the training of the MPO classifier, we use the same setup and hyperparameters as for the MPS classifier.

Performance of the tensor-network classifiers. In figure 10 we compare the performance of the MPS and MPO classifiers and an SVM classifier on the four datasets, when the input image is encoded using multiple patches (see section 2). Both the training (top row) and validation (bottom row) accuracies are shown. We show results for the tensor-network classifiers initialized with two different strategies before training: random initialization (dashed lines, crosses and down-facing triangles) and a warm start through pretraining (solid lines, pluses and up-facing triangles). The results for the MPO classifier are shown in blue, for the MPS classifier in orange, and for the SVM classifier as green stars.

On the MNIST dataset, all classifiers achieve a high accuracy: for a single patch it is already above 90%, and grows above 96% when using multiple patches. For all classifiers both training and validation accuracies increase when splitting the image into increasingly more patches. The MPO classifier outperforms the other models when encoding the full image as a single patch, however, for more patches they perform similarly well. For more patches, we can also see that the warm-start initialized tensor-network classifiers tend to perform slightly better than the randomly initialized ones. The overall similarity between the achieved training and validation accuracies suggests that none of the classifiers are overfitting strongly.

The slightly more complex Fashion-MNIST dataset leads to a decreased accuracy of all classifiers, but their overall performance is still good with a validation accuracy of around 89% on 16 patches. Compared with the results on the MNIST dataset, the difference in performance between the MPS and MPO classifiers is less pronounced. Still, we observe the general trends that more patches tend to improve performance, and that for many patches the warm-start initialization outperforms random initialization. On a single patch the MPO classifier performs best, but on more patches the SVM achieves the highest validation accuracy,

reaching $90.4 \pm 0.2\%$ on sixteen patches—considering its training accuracy, however, it also begins to overfit significantly.

Moving to the more realistic CIFAR-10 dataset, the first of the datasets to consist of color images, we see the accuracy of all classifiers drop considerably compared to the two previous datasets. Nonetheless, we observe some similar trends as before: the MPO classifier outperforms the other models for a small number of patches, but this difference becomes less significant when using more patches. For sixteen patches the SVM classifier eventually beats the MPO classifier with an accuracy of $56.7 \pm 0.1\%$; however, it also drastically overfits when compared with its training accuracy of $95.6 \pm 0.1\%$. Unlike before, increasing the number of patches does not improve performance for all models, for the randomly initialized tensor-network classifiers the validation accuracy stagnates or even drops when changing from four patches to sixteen.

Finally, we consider the Imagenette dataset, which includes color images with a higher resolution than the other datasets. On this dataset, the performance of all classifiers is quite poor. Previously, the MPO classifiers using either initialization technique outperformed the MPS classifiers (at least for a small number of patches), but now both warm-started tensor-network classifiers outperform the randomly initialized ones. Also, while the accuracy improves when going from one to four patches, it does not further improve when using sixteen patches: it stagnates for the SVM and warm-started MPS classifiers, and even decreases for the randomly initialized MPS and both MPO classifiers. The best accuracy is achieved by both the SVM classifier with $50.0 \pm 0.3\%$ and the warm-started MPO classifier for four patches with $49.5 \pm 0.6\%$; however, the SVM again overfits drastically more than the MPO classifier.

Figure 11 shows analogous results when a product state of multiple copies of the image, rather than multiple patches, is provided as an input to the tensor network (see section 2). The top row shows the training accuracy and the bottom row the validation accuracy, as the number of copies of the image is increased from one to four. (Note that the results for a single copy are the same as for a single patch in figure 10). The results for the MPO classifier are again shown in blue, the results for the MPS classifier again in orange. Warm-started and randomly initialized tensor-network classifiers are distinguished by solid and dashed lines, respectively, and different marker symbols. The results for the SVM classifier are shown as green stars.

As in the case of the patched encoding, all classifiers achieve a high accuracy on the MNIST dataset, although the best-performing models with just under 98% accuracy remain below the accuracy of the best-performing models with the patched encoding that achieved just over 98%. We observe that introducing more copies increases the performance for all models. The MPO classifiers outperform the MPS classifiers with the same initialization, and we see that for more copies the warm-started tensor-network classifiers outperform their randomly initialized counterparts. In contrast to what we saw for the patched encoding, here the SVM classifier performs worse than the tensor-network classifiers, even though for more copies this difference diminishes.

On the Fashion-MNIST dataset, the performance of all classifiers slightly drops. The resulting accuracies are comparable to those obtained using the patched encoding, but the best-performing models remain below the accuracy of the best-performing models on the patched dataset. We still see the trend that the accuracy of the classifiers increases as the number of copies is increased, however for three or more copies this saturates and all models perform similarly well.

Compared to the first two datasets, the accuracy on the CIFAR-10 dataset is significantly lower, but remains comparable to the accuracy on the patch-encoded CIFAR-10 dataset. Similar to before, we see the trend that an increasing number of copies tends to improve the performance of the classifier. The MPO classifier outperforms the MPS classifier; for the MPO classifier the warm-starting works better than random initialization, while for the MPS classifier both work similarly well. In contrast to the patched encoding, the SVM is surprisingly the worst performing classifier out of the three models.

On the higher-resolution Imagenette dataset, the accuracy drops even further and remains below 50%, as it similarly did for the patched encoding. Increasing the number of copies of the input image only moderately increases the performance and quickly saturates. For this dataset we see that the warm-started tensor-network classifiers outperform the randomly initialized ones, and the SVM classifier lies somewhere in between. This is also the only dataset for the multi-copy encoding where the SVM classifier very strongly overfits, reaching testing accuracies twice as large as the validation accuracy—for the patched encoding strong overfitting of the SVM classifier was also visible on the Fashion-MNIST and CIFAR-10 datasets.

Overall, we see that it is important to test these classifiers on a range of datasets, including on realistic datasets. Results can change when moving away from toy datasets, exemplified here by the drastic decrease of performance when moving from the rather simple MNIST and Fashion-MNIST datasets to the more realistic CIFAR-10 and Imagenette datasets.

Some general observations seem to hold on all datasets: adding nonlinearities to the classifiers, either by increasing the number of patches or the number of copies, tends to increase performance—but there are diminishing returns for more patches or copies. The benefit of the nonlinearity is also exemplified by both

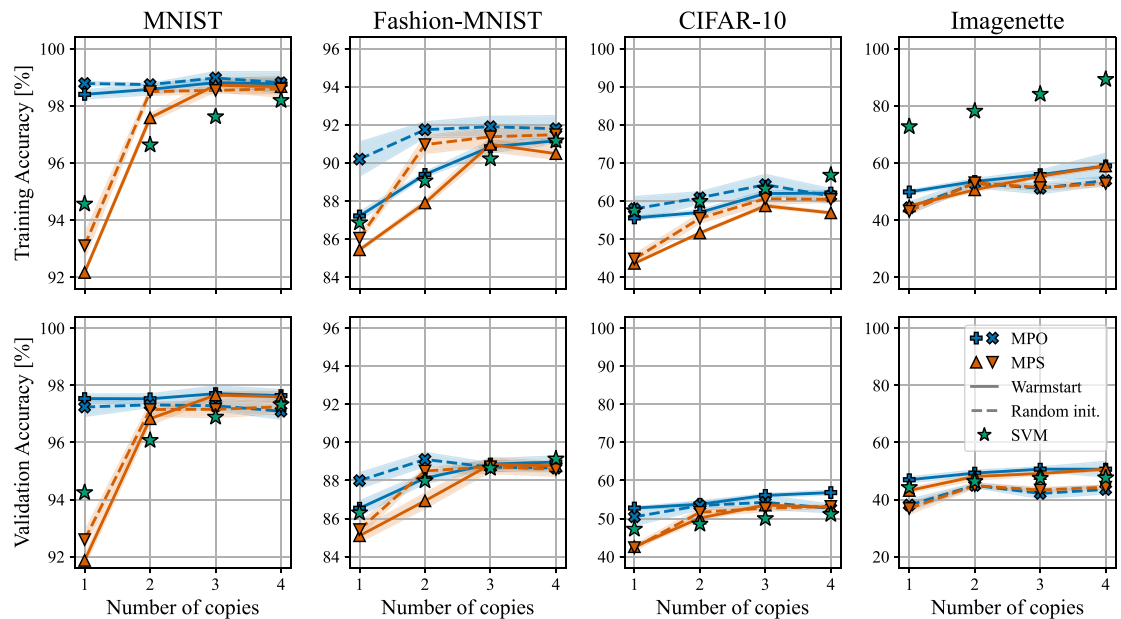


Figure 11. Accuracy of the tensor-network and SVM classifiers versus the number of copies used in the encoding. Both the training (top row) and validation accuracy (bottom row) are shown on the four datasets MNIST, Fashion-MNIST, CIFAR-10 and Imagenette. Results are shown for the MPS (orange) and MPO (blue) classifiers, for both the warm start initialization (solid lines) and random initialization (dashed lines). The results for the SVM are shown as green stars. The baseline accuracy for random guessing is 10%. The error bands represent the standard deviation from five-fold cross-validation.

the SVM and MPO classifiers outperforming the MPS classifiers on the patch-encoded datasets, as the latter can only construct linear functions of the input state amplitudes while the former can construct nonlinear quadratic functions. This performance difference is most pronounced for a small number of patches, as with an increasing number of patches also the MPS classifier becomes increasingly nonlinear with respect to the original pixel values. On the multi-copy encoded datasets the SVM only outperforms the MPS classifier on a single patch, where the same argument holds. For more than one copy, also the MPS classifier has access to quadratic functions of the input state, and higher order polynomials seem to be not much more helpful as the accuracy of all classifiers does not increase much beyond two copies of the input state. Viewing the classifier as a polynomial of the input state with its degree set by the number of copies is equivalent to the more common perspective of the output of a classifier as a Fourier series with respect to the input data⁵, where the range of accessible Fourier frequencies grows with the number of copies [13]. So, naively, one would expect the expressivity to directly grow with the number of copies of the state. However, this is not necessarily the case as, depending on the structure of the circuit ansatz, the coefficients of the resulting Fourier series can be quite constrained [125–127].

On the larger models that require many qubits—either because there are many patches or copies of the image, or because the images are in color and have a higher resolution—we observe that the warm-start initialized tensor-network classifiers tend to outperform the randomly initialized ones. This might be because for models with many qubits the output of the classifier is extremely sensitive to the norm of the tensors forming the tensor network: the overlap with an input state either vanishes or explodes exponentially if they are too small or too large. While our specific choice of random initialization fixes the standard deviations so that the output is not exponentially small or large, after a few steps of the optimization algorithm we can no longer make such a statement. The pretraining initializes the classifier in a more controlled way, which seems to place it in a more well-behaved region of the cost landscape.

A final observation is that, while all classifiers perform better on the training than on the validation set, only the SVM overfits very strongly. This is less pronounced on the multi-copy encoded datasets than on the patch-encoded datasets, but visible for both approaches and most significant as the number of qubits becomes large. As discussed in the section on quantum kernel methods, this is somewhat expected from the way this classifier is constructed. In particular, as the number of qubits grows, distinct states become increasingly more orthogonal and so the quantum kernel function approaches a Kronecker delta, which

⁵ The output of the classifier is a polynomial of the state amplitudes, which consist of products of $\cos(\pi x_j/2)^{2k}$ or $\sin(\pi x_j/2)^{2k}$, where k can be up to the number of copies of the input state. Equivalently, when viewed as a function of the data components x_j , these terms are part of a Fourier series including terms up to $e^{\pm i 2k \frac{\pi}{2} x_j}$ with frequency $2k$.

would lead to the SVM being able to classify the training set perfectly but being unable to predict anything in the validation set. The explicit approaches to the classifier do not necessarily have this problem.

4.3. Performance on the compressed datasets

For the performance evaluation on the compressed datasets, we use the model configurations of the VQCs, the SVM, and tensor-network classifiers that achieved high accuracies on average in earlier experiments. We focus on the multi-copy encoding over the patched encoding since approximating only small patches should be easier than approximating the full images, and we are precisely interested in the error arising from the imperfect approximation. Specifically, we use an SVM with three copies of the input state, an MPO classifier with three copies and warm-start initialization, and a nonlinear VQC with four layers trained on a single copy. All models are trained using the same hyperparameters as in the previous experiments. In addition, we also train a classical CNN on the compressed datasets to see whether any loss of accuracy is due to information loss during the compression, or because the quantum classifiers are sensitive to the compression-induced distortion of the input state.

CNNs. For a classical baseline, we compare the classification performance of the quantum models with the performance of a CNN [128]. CNNs are a well-established model for processing image data, leveraging local connectivity and parameter sharing to extract spatial hierarchies of features. While recent models like vision transformers [129] have gained popularity, CNNs still demonstrate superior performance on small datasets when transfer learning is not employed.

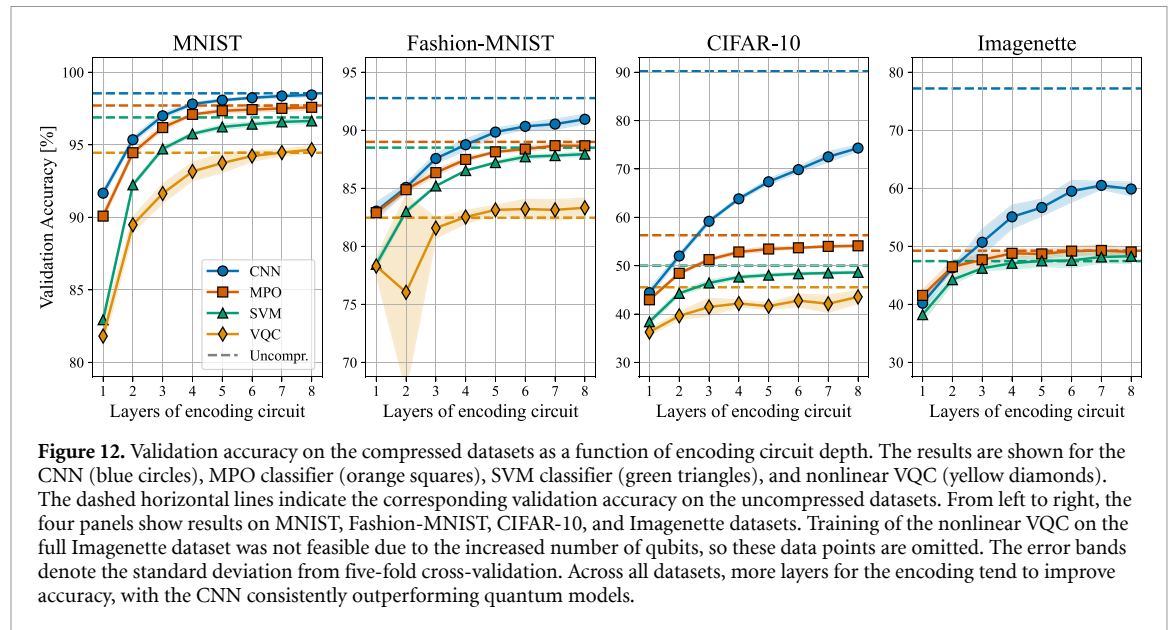
To restore spatial locality, we reshape the encoded states and invert the hierarchical ordering. For the grayscale images of the MNIST and Fashion-MNIST datasets this results in inputs of shape $(32, 32, 2)$, and shapes $(32, 32, 8)$ and $(128, 128, 8)$ for the color images of the CIFAR-10 and Imagenette datasets. The last dimension corresponds to the amplitudes of the quantum state encoding the color values of each pixel—two amplitudes from the single color qubit used for the grayscale encoding and eight amplitudes from the three color qubits used for the color encoding. Since no preprocessing is used for training the quantum classifiers, no transformations are applied to the input; all values are real and relatively small as a result of the full quantum state normalization.

The CNN architecture resembles the one of SimpleNet [103] and we only adjust the number of input channels from the standard three (RGB images) to the number of basis states of the color qubits (i.e. two for grayscale and eight for color images). For all configurations this results in a parameter count of about 5.5 million. The overall architecture comprises multiple convolutional blocks, each consisting of a convolutional layer, batch normalization, and ReLU activation. Additionally, intermediate max pooling and dropout layers are applied. Like the original study, we use Adadelta [130] as the optimization algorithm and Xavier uniform initialization [131] for the convolutional layers. We use a multi-step learning rate scheduler, where the learning rate is decreased by a factor of 0.1 at predefined milestone epochs. Given that inputs are represented as quantum states rather than RGB color images, we performed a hyperparameter optimization. Like in the original publication, we performed the optimization only for CIFAR-10 (on uncompressed states) and used the same hyperparameter configuration for all four datasets. We optimized the learning rate, dropout probability, and weight decay. The best-performing configuration used a learning rate of 0.17, a dropout probability of 0.3, and a weight decay of $4 \cdot 10^{-4}$. For this purpose, we sampled 200 hyperparameter configurations and employed the Asynchronous Successive Halving Algorithm (ASHA) scheduler [132] in connection with the hyperband optimizer [133]. These tools incorporate early stopping and Bayesian-based hyperparameter optimization.

Performance on the compressed datasets. Figure 12 shows the validation accuracy of four classifiers—the CNN (blue circles), MPO classifier (orange squares), SVM classifier (green triangles), and nonlinear VQC (yellow diamonds)—on the compressed versions of the MNIST, Fashion-MNIST, CIFAR-10, and Imagenette datasets as a function of the number of layers of the image-encoding circuit. Shaded areas denote one standard deviation from five-fold cross validation. The dashed lines show the performance on the uncompressed dataset as a reference value.

On the compressed MNIST dataset, as the number of layers of the encoding circuit increases, all models quickly reach a similar validation accuracy as they did on the uncompressed dataset. When using around eight layers for the encoding circuit, all models achieve the same accuracy as on the uncompressed dataset.

Also on the Fashion-MNIST dataset all models achieve a similar accuracy on the compressed and uncompressed datasets when a moderate number of layers is used for the image encoding circuit. For the MPO and SVM classifiers, the same accuracy as on the uncompressed dataset is reached at around seven to eight layers; for the nonlinear VQC the compression acts as a regularization and when the encoding circuit uses five or more layers it even helps to outperform the nonlinear VQC trained on the uncompressed dataset.



Only the CNN does not reach the same accuracy as on the uncompressed dataset, but even this worse performance achieves significantly higher accuracies than all other models (both those trained on the compressed or uncompressed datasets).

The CIFAR-10 dataset is the first dataset where the quantum classifiers start to struggle on the uncompressed dataset. We also see this in the performance on the compressed dataset, which with an increasing number of layers of the encoding circuit quickly reaches a similar level as on the uncompressed dataset but never exactly the same accuracy. The CNN does not struggle with the uncompressed CIFAR-10 dataset and reaches 90% accuracy. However, it is more affected by the compression of the dataset than the quantum classifiers, and it does not reach similar accuracies on the compressed version. Still, the accuracies reached on the compressed dataset are much larger than any of the other models, even when those are trained on the uncompressed dataset. The dip in performance of all models on the compressed CIFAR-10 dataset may be partially due to these images having quite a lot of detail for their comparably low resolution. Preserving all of these minute details is hard during the compression, but may be necessary for correct classification.

On the Imagenette dataset the MPO and SVM classifiers reach the same (or larger) accuracy as on the uncompressed dataset when the dataset is compressed with more than four layers of the encoding circuit. Results for the nonlinear VQC are omitted as training on the full-scale Imagenette dataset was not feasible due to the increased number of qubits. The CNN does perform worse on the compressed dataset than it does on the uncompressed one, but even the accuracies achieved on the compressed dataset are significantly better than any of the other models' accuracies on either the compressed or uncompressed datasets.

Overall, we see that the CNN outperforms all other models—whether they are trained on the compressed or uncompressed datasets. With the CNN we do not quite reach the state-of-the-art levels reported in section 3.3; this is not unexpected, as we feed the normalized quantum states into the CNN and not the images directly. For the Imagenette dataset, we further crop and resize the images which may additionally reduce the accuracy.

The purpose of the CNN comparison is not to achieve state-of-the-art performance or to directly evaluate whether CNNs outperform the quantum classifiers; this outcome is expected, given that CNN architectures and training procedures have been refined over decades. Furthermore, the CNN models used here have the highest parameter count among all models considered. Rather, the objective is to assess how much information is preserved during compression: specifically, whether the essential features required for classification remain present in the quantum states after encoding the image data into circuit form.

While the CNN is affected the most by the compression out of all classifiers, this is most likely because we directly optimize the fidelity (related to the usual two-norm) when compressing the states. The fidelity is directly related to the kernel functions that are evaluated for the SVM classifiers, and to the expectation values measured with either the quantum circuit classifiers or the tensor-network classifiers. Thus, optimizing the fidelity leads to directly minimizing deviations in the outputs of these classifiers. Conversely, the CNN is not directly related to the two-norm as it is a highly nonlinear function. While optimizing the

fidelity should result in similar enough compressed images that the classifier can still recognize them, there are no guarantees that the deviation of the CNN output is minimally affected like for the quantum models.

Nonetheless, we see that the CNN trained on the compressed datasets clearly outperforms all other models, even those trained on the uncompressed datasets. Thus, we see that the quantum models in general are not limited by the amount of information present in the input state, but rather by the way they process that information. This highlights the need for new ideas and architectures for quantum models, which can better extract the available information from the quantum state. As we have seen in the previous subsections, exploring the integration of nonlinearities into these models—either during the processing of the quantum circuit or before that in the input state—may be a fruitful direction.

5. Conclusion

Loading classical data on a quantum computer is a major bottleneck for many QML algorithms. Here, we have investigated a method that allows to efficiently load classical image data, and other data with a similar structure. We considered two different encodings, the FRQI for greyscale images and the MCRQI for color images. These encodings lead to lowly-entangled quantum states, which are well-captured by MPS. This, in turn, means that low-depth MPS-inspired quantum circuits can approximate the encoded image data with a high fidelity. We have also presented an optimization algorithm capable of finding these low-depth quantum circuits.

We have applied this algorithm to four standard machine learning datasets, the MNIST, Fashion-MNIST, CIFAR-10 and Imagenette datasets, to test the effectiveness of our approach. We found that these low-depth circuits indeed capture the encoded images with a high fidelity. The fidelity can be systematically increased by growing the number of layers in the circuit, without signs of the fidelity saturating. Further, even as the image resolution is increased, we found that a fixed number of layers of the circuit was able to capture the state, suggesting that this method also scales to higher-resolution image data. The optimized quantum circuits for loading the four image datasets are available via PennyLane at <https://pennylane.ai/datasets/collection/low-depth-image-circuits>. We invite researchers to use them to run their own benchmarks.

Additionally, we presented benchmark results from training quantum classifiers on these datasets, first on the uncompressed and then on the compressed version. These classifiers include quantum kernel methods, VQCs, and tensor-network-based classifiers. On the uncompressed dataset, we observed that the classifiers perform well on the simple MNIST and Fashion-MNIST datasets, but become less effective on the more complex CIFAR-10 and Imagenette datasets. Shifting from the linear to the nonlinear VQC, or employing the patched or multi-copy encoding to introduce nonlinearities into the tensor-network classifiers, showed promising results for enhancing the models' performance, including on the more complex datasets. Ultimately, however, it is not enough to bring them in line with state-of-the-art performance.

On the compressed dataset the quantum classifiers reach roughly the same validation accuracy, even if the circuits encoding the images only consist of a few layers—in some cases the accuracy is a bit worse, but sometimes, surprisingly, the accuracy even increases on the compressed dataset. We also trained a classical CNN to ensure that the relevant information needed for classifying the images survived the compression step. The CNN achieves a great performance on the uncompressed data, and even on the compressed complex datasets it performs significantly better than the quantum models. This suggests that the quantum models are not limited by the amount of information available in the input state—that information is preserved by the encoding and compression steps. Rather, the quantum models have insufficient capabilities to separate the complex datasets. Indeed, a consistent observation across our benchmarks was that the models perform better as they get access to nonlinearities. Thus, a promising future research direction might be to focus on designing quantum architectures that can effectively incorporate nonlinearities.

Also feature maps can give rise to nonlinearities. Here we have focused on the FRQI and MCRQI for mapping the image data to quantum states, and used the patched or multi-copy encoding to achieve more complex feature maps. It would be interesting to see if there are other mappings from the classical data to quantum states, that realize the necessary nonlinearities but remain lowly entangled such that they can be efficiently loaded in the same way.

Finally, the compressed datasets are readily available via PennyLane. This enables researchers to benchmark novel ideas on the same collection of datasets. We hope that this facilitates comparison among different QML models, and allows to identify promising methods that also work as the complexity of the dataset is increased. Instead of benchmarking QML models, these datasets could also be used to benchmark quantum hardware by running established QML algorithms on them.

Data availability statement

The data that support the findings of this study are openly available at the following URL/DOI: <https://pennylane.ai/datasets/collection/low-depth-image-circuits>.

Acknowledgment

The authors thank Diego Guala and Ben Lau from Xanadu for their support regarding the distribution of the data, and Lukas Lechner for generously granting permission to use their photograph in figure 1(b). B J, F P and C A R thank Kevin Shen and Elvira Shishenina for their collaboration on related previous projects. B J thanks Sheng-Hsuan Lin for interesting discussions about the optimization of quantum circuits. F P acknowledges the support of the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy EXC-2111-390814868, the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation program (Grant Agreement No. 771537), as well as the Munich Quantum Valley, which the Bavarian state government supports with funds from the Hightech Agenda Bayern Plus. Additionally, the work received support from NERSC (ERCAP0032477/ERCAP0029512: 'Characterization and Middleware for Hybrid Quantum-HPC Applications') and the Bavarian State Ministry of Economic Affairs (BenchQC project, Grant DIK0425/03).

Code availability

The four datasets are available via PennyLane at <https://pennylane.ai/datasets/collection/low-depth-image-circuits>. The code for circuit optimization and benchmarking of the classifiers is available at <https://github.com/fkiwit/low-depth-image-circuits>.

Appendix A. Other color image encodings

In the main text we solely focused on the MCRQI to encode color images, but, of course, it is not the only way to encode the color information of an image into the amplitudes of a quantum state. Here, we consider and compare other similar encoding strategies. We also consider different ways to index the pixels in the image, as in the main text we claimed that the Z-order we used leads to the smallest entanglement entropies. We find that all encodings with all indexing variants follow an area law for the entanglement entropy, i.e. the bipartite entanglement entropy of these states does not grow with the image resolution (which sets the number of qubits). The MCRQI with hierarchical indexing on average has the smallest entanglement entropy in our numerical results, suggesting that they can be represented by tensor networks most effectively, which is why we focus on this encoding in the main text.

All states we discuss are of the same form as in the main text, see equation (1), that is they can be written as

$$|\psi(\mathbf{x})\rangle = \frac{1}{\sqrt{2^n}} \sum_{j=0}^{2^n-1} |c(\mathbf{x}_j)\rangle \otimes |j\rangle. \quad (14)$$

For the different encodings, only the state of the color qubits $|c(\mathbf{x}_j)\rangle$ will be different. We will also consider three different indexing variants, which change how the pixels are labeled by the index j . The simplest variant is when the pixels are counted left to right within each row, and row by row from top to bottom [44, 60, 61]; in the hierarchical variant that we use in the main text the pixels are indexed by the Z-order (see figure 1(a)) [14, 44, 60, 61]; and, finally, we consider a snake ordering where rows are ordered top to bottom as in the row-by-row indexing, but every other row is traversed in the opposite direction [43, 44].

Taking the FRQI for encoding grayscale images as motivation, as a first example we could just consider a state with three color qubits that are simply a tensor product of the single-color-qubit states used in the FRQI, each encoding one of the three different RGB channels. Then, the state of the color qubits is

$$\begin{aligned} |c(\mathbf{x}_j)\rangle = & \left(\cos\left(\frac{\pi}{2}x_j^R\right)|0\rangle + \sin\left(\frac{\pi}{2}x_j^R\right)|1\rangle \right) \\ & \otimes \left(\cos\left(\frac{\pi}{2}x_j^G\right)|0\rangle + \sin\left(\frac{\pi}{2}x_j^G\right)|1\rangle \right) \\ & \otimes \left(\cos\left(\frac{\pi}{2}x_j^B\right)|0\rangle + \sin\left(\frac{\pi}{2}x_j^B\right)|1\rangle \right), \end{aligned} \quad (15)$$

and we call this encoding $\otimes$ -multi FRQI.

The state above uses all eight different basis states, and once it is entangled with the address register the bond dimension across that bond will generically be maximal. By taking a direct sum of the different FRQI components, instead of the tensor product, we get only six contributing basis states, limiting the bond dimension of this bond. This sets the state of the color qubit to be

$$\begin{aligned} |c(\mathbf{x}_j)\rangle = & \frac{1}{\sqrt{3}} \left(\cos\left(\frac{\pi}{2}x_j^R\right) |000\rangle + \sin\left(\frac{\pi}{2}x_j^R\right) |100\rangle \right. \\ & + \cos\left(\frac{\pi}{2}x_j^G\right) |001\rangle + \sin\left(\frac{\pi}{2}x_j^G\right) |101\rangle \\ & \left. + \cos\left(\frac{\pi}{2}x_j^B\right) |010\rangle + \sin\left(\frac{\pi}{2}x_j^B\right) |110\rangle \right), \end{aligned} \quad (16)$$

and we call this encoding $\oplus$ -multi FRQI.

And finally, for completeness, the MCRQI amounts to adding another α channel to the previous encoding, but fixing all $x_j^\alpha = 0$. The state of the color qubits is then

$$\begin{aligned} |c(\mathbf{x}_j)\rangle = & \frac{1}{2} \left(\cos\left(\frac{\pi}{2}x_j^R\right) |000\rangle + \sin\left(\frac{\pi}{2}x_j^R\right) |100\rangle \right. \\ & + \cos\left(\frac{\pi}{2}x_j^G\right) |001\rangle + \sin\left(\frac{\pi}{2}x_j^G\right) |101\rangle \\ & + \cos\left(\frac{\pi}{2}x_j^B\right) |010\rangle + \sin\left(\frac{\pi}{2}x_j^B\right) |110\rangle \\ & \left. + \cos\left(\frac{\pi}{2}x_j^\alpha\right) |011\rangle + \sin\left(\frac{\pi}{2}x_j^\alpha\right) |111\rangle \right), \end{aligned} \quad (17)$$

as in the main text—see equation (3).

The three different encodings are compared in figure 13. As a representative dataset, we consider the 174 images in the Imagenette dataset [52] that originally have a higher resolution than 1024×1024 pixels. The figure shows the maximal entanglement entropy over all bipartitions of the state into two contiguous halves, averaged over all images in the dataset, plotted against the number of qubits needed to represent the images at increasingly higher resolutions. The different colors denote the different encodings, i.e. blue shows the data for the $\otimes$ -multi FRQI, orange the data for the $\oplus$ -multi FRQI and green the data for the MCRQI. The marker shapes and line styles denote the different ways the pixels are indexed; circles connected by a solid line show the data for row-by-row indexing, pluses connected by a dashed line the data for hierarchical indexing and crosses connected by a dotted line the data for snake indexing. The shaded areas show the 25th–75th percentiles for the different image encodings using the hierarchical indexing; we omit the shaded areas for the data points corresponding to the other indexing variants for visual clarity, the results would look similar. As a reference value, we also plot the half-chain entanglement entropy of states with amplitudes that are sampled randomly from a normal distribution with zero mean and a variance of one before normalization. The average of 200 realizations is marked by black stars, the 25th–75th percentiles are shaded in black. The black dash-dotted line shows a linear fit to the data, illustrating the generic case where the entanglement entropy of quantum states grows linearly with the subsystem size [134].

From the numerical data, we can see that the states encoding image data behave quite distinctly from random or typical quantum states. In particular, we can see that for all encodings and indexing variants the bipartite entanglement entropy seems to saturate to a constant value as the number of qubits is increased, in contrast to the linear growth for random states. We can also see that, on average, the MCRQI states are the least entangled out of the three encodings, and the $\oplus$ -multi FRQI states are less entangled than the $\otimes$ -multi FRQI states. The choice of indexing makes less of a difference, but we can still see that for high resolutions there is a tendency that the hierarchical encoding leads to more lowly-entangled states than the row-by-row indexing, which leads to more lowly-entangled states than the snake indexing. Overall, the combination which on average leads to the smallest entanglement entropies, and thus should be the most compressible, is the MCRQI encoding with hierarchical indexing—this is what we use in the main text.

Appendix B. Compiling the circuits to the Clifford+ T gate set

Apart from the scaling of the infidelity with an increasing number of CNOTs, as shown in figure 5, one might also be interested in the infidelity as a function of the number of T gates when compiling the circuits into the Clifford+ T gate set suitable for a fault-tolerant quantum computer. There, Clifford operations are effectively free, and the resource overhead is dominated by the T gates, which have to be implemented, e.g. via a costly magic-state distillation [135]. Here we consider this fault-tolerant setup, and show the resulting T -gate counts in figure 14.

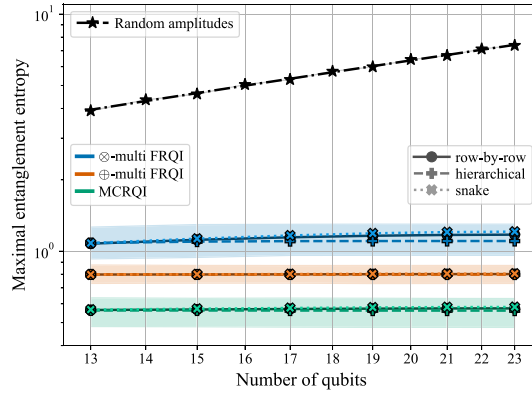


Figure 13. Scaling of the entanglement entropy for different color image encodings. The plot shows the scaling of the maximal entanglement entropy over any bipartition of the state into two contiguous halves with the number of qubits in the state. The quantum states are obtained from encoding the 174 highest-resolution images of the Imagenette dataset [52] with the three different image encodings— $\otimes$ -multi FRQI (blue), $\oplus$ -multi FRQI (orange) and the MCRQI (green)—and the three different indexing variants—row-by-row (circles with solid lines), hierarchical (pluses with dashed lines) and snake (crosses with dotted lines)—discussed in the text. The shaded areas show the 25th–75th percentiles for the different encodings using the hierarchical indexing. For reference, we also show the average half-chain entanglement entropy for 200 states whose amplitudes were drawn from a normal distribution before normalization as black stars. The black shaded area shows the 25th–75th percentiles and the dash-dotted line shows a linear fit to the data.

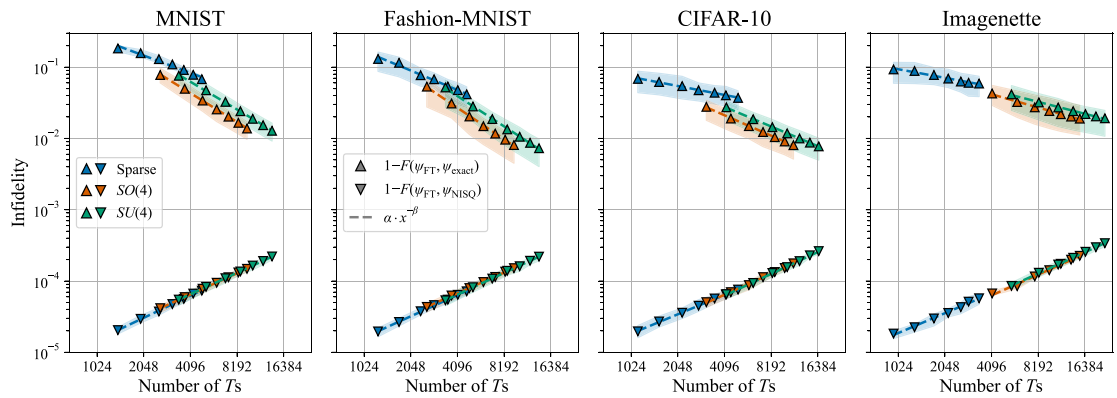


Figure 14. Scaling of the infidelity with an increasing number of T gates. We encode 100 randomly selected images (10 per class) from the MNIST, Fashion-MNIST, CIFAR-10 and Imagenette datasets as discussed in section 2. The upwards-facing triangles show the average infidelity between the exactly encoded states and their quantum circuit approximations compiled into the Clifford+ T gate set, the downwards-facing triangles show the average infidelity between the compiled quantum circuits and the same quantum circuits pre-compilation. Both are plotted against the number of T gates in the circuit after compilation. Blue markers show the data for circuits using sparse two-qubit gates, orange markers for those using special orthogonal two-qubit gates and green markers for those using general unitary two-qubit gates. The shaded areas show the 25th–75th percentiles of the infidelities, the variance of the T -gate count between different images is negligible. The dashed lines show the results of fitting an algebraic decay or growth $f(x) = \alpha x^{\mp\beta}$ to the data. The fitted values, including their uncertainties, are given in tables 3 and 4.

To compile the circuits into the Clifford+ T gate set, we note that we already have the circuits in the form of CNOT gates, which are Clifford gates, and single-qubit gates. For the circuits using the sparse or special orthogonal two-qubit gates, the single-qubit gates are simply y -rotations. For the circuits using general two-qubit gates also general single-qubit gates appear, which can further be decomposed into single-qubit rotation gates. Up to a basis transformation composed of Clifford operations, these single-qubit rotations are equivalent to a z -rotation. Thus, we only have to consider the decomposition of these single-qubit z -rotations, for which there exist very efficient decomposition algorithms [136].

We use the Python implementation [137] of the Gridsynth algorithm [136] to transform the R_z gates into a sequence of single-qubit Clifford and T gates. The typical number of T gates returned by this algorithm to reach a precision ε is $3 \log_2(1/\varepsilon) + O(\log \log(1/\varepsilon))$. For our numerical results we use a precision of $\varepsilon = 10^{-3}$ to ensure that the combined approximation errors arising from the gate decompositions play a subleading role compared to the approximation error from the circuit optimization.

Figure 14 shows the infidelities of the circuits compiled with the Clifford+ T gate set against the mean number of T gates used. The data points in the upper half of the plot shown as upwards-facing triangles correspond to the infidelity of the compiled circuit with the uncompressed quantum state of the image, and

the data points in the lower half shown as downwards-facing triangles correspond to the infidelity of the compiled circuit with the quantum circuit pre-compilation containing arbitrary-angle rotation gates. As in the main text, blue markers show the data for the circuits optimized using the sparse gates, orange markers for circuits optimized with special orthogonal gates and green markers for circuits optimized with general unitary gates. The shaded areas show the 25th–75th percentiles of the infidelity. In comparison, the variance in the required number of T gates for the different images is negligible and thus we omit plotting it. The dashed lines show the result of an algebraic fit to the data, the obtained fitted parameters and their uncertainties are summarized in tables 3 and 4.

Overall, the results for the infidelities with respect to the uncompressed states as a function of the T -gate count looks similar to those for the CNOT count (see figure 5). Even the exponents of the power laws obtained from fitting to the data are very close when plotting against the CNOT count or T -gate count (see tables 1 and 3). Ignoring the initial layer of single-qubit gates, sparse and special orthogonal gates require two single-qubit rotations per CNOT, and general unitary gates require three per gate. Assuming the same mean number of T gates for the three different gate types, we expect the same relative behavior between sparse and special orthogonal gates as when plotting against the CNOT count. Only the results for the general unitary gates would be expected to appear slightly shifted towards larger values. At a closer look, however, the mean number of T gates is not the same for all gate types, as for the sparse gates we observe that the total number of T gates actually decreases with image resolution (i.e. going from MNIST to Imagenette), while it grows for the other two gate types. This artifact arises because, in circuits using sparse gates, many single-qubit gates acting on the few least-significant address qubits in the MCRQI are nearly identity operations and are therefore compiled as identity gates. We could remove this artifact by setting the compilation precision to a lower value of $\varepsilon = 10^{-5}$, but already at a precision of $\varepsilon = 10^{-3}$ the error arising from the compilation is orders of magnitudes smaller than the error that stems from the circuit approximation error. The low approximation error arising from the compilation to the Clifford+ T gate set can be seen from the data in the lower half of the plot: for a circuit with two layers, the infidelity of the compiled circuit with its pre-compiled counterpart is roughly three orders of magnitude lower than the infidelity of the compiled circuit with its target state, and for the eight-layer circuits this difference is still roughly two orders of magnitude.

We can speculate on why so many near-identity gates appear for the circuits using sparse gates, more so than for the other two gate types. Note that most of these near-identity gates appear when acting on the least significant bits of the address qubits, which control the finest length scales of the image (see figure 1(a)). For a high-resolution image, a good approximation to the image might be a coarser version obtained by blocks of pixels with uniform color. This corresponds to completely disentangling the qubits corresponding to the few least-significant bits and preparing them in an equal-weight superposition. The finer details are then obtained by weakly entangling these qubits with the address qubits corresponding to more significant bits and the color qubits. We can imagine that after the first layer of the circuit is initialized during the optimization algorithm, such a state is prepared, where the qubits corresponding to the least-significant bits are close to a product state made of equal-weight superpositions and they are only weakly entangled with the rest of the system. The next few layers of the circuit initialized during the optimization should then only marginally increase the correlations of the least-significant bits with the rest of the system, keeping the overall weakly-entangled structure intact. However, since the sparse gates are just a single-CNOT with single-qubit gates, they are always a maximally entangling gate that generically increases the entanglement entropy significantly. Thus, it might turn out best to keep the single-qubit gates very close to an identity, so that many of the CNOTs act on the state $|0\rangle$ and do not increase the entanglement entropy. Conversely, special orthogonal and general unitary gates can locally be tuned to the identity by choosing suitable single-qubit gates, which should allow for a finer control of how much entanglement and what kind of correlations are added to the least significant bits.

Appendix C. Fit parameters

Here we give the detailed fitted parameters for the scaling of the infidelities of the circuit approximations plotted in the main text in section 3. The parameters obtained from fitting an algebraic decay to the infidelities as a function of the CNOT count as shown in figure 5 are listed in table 1. The fitted parameters for the Gompertz functions that describe the scaling of the infidelities with image resolution in figure 6 are summarized in table 2.

We also show the detailed obtained parameters from fitting the infidelities as a function of the T count as considered in appendix B. For the infidelities of the circuits compiled into Clifford and T gates with respect to the uncompressed quantum states encoding the image data as shown in figure 14(upper half, upwards-facing triangles), the obtained parameters are listed in table 3. The fitted parameters for the infidelity between the compiled circuits and the circuits pre-compilation also shown in figure 14(lower half, downwards-facing triangles) are listed in table 4.

Table 1. Parameters obtained from fitting an algebraic decay $f(x) = \alpha x^{-\beta}$ to the infidelities of the circuit approximations as a function of the number of CNOTs in the circuit. We show results for states encoding images from the four datasets MNIST, Fashion-MNIST, CIFAR-10 and Imagenette, and for circuits using sparse, special orthogonal and general unitary gates. These parameters correspond to the algebraic decays shown as dashed lines in figure 5.

Setting	α	β
MNIST		
Sparse	1.80 ± 0.48	0.812 ± 0.069
SO(4)	8.46 ± 0.77	1.263 ± 0.020
SU(4)	16.27 ± 1.60	1.304 ± 0.020
Fashion-MNIST		
Sparse	3.05 ± 1.04	1.02 ± 0.088
SO(4)	8.71 ± 0.41	1.382 ± 0.010
SU(4)	18.16 ± 1.45	1.436 ± 0.016
CIFAR-10		
Sparse	0.47 ± 0.12	0.506 ± 0.063
SO(4)	0.91 ± 0.03	0.902 ± 0.006
SU(4)	1.44 ± 0.02	0.929 ± 0.003
Imagenette		
Sparse	0.38 ± 0.06	0.400 ± 0.035
SO(4)	0.42 ± 0.01	0.548 ± 0.005
SU(4)	0.54 ± 0.04	0.566 ± 0.013

Table 2. Parameters obtained from fitting a Gompertz function $f(x) = \alpha \exp(-\beta \exp(-\gamma x))$ to the scaling of the infidelities of the circuit approximations with increasing image resolution. We show results for circuits using special orthogonal gates with $d \in \{1, 2, 3, 4\}$ layers approximating states encoding high-resolution images from the Imagenette dataset, and for a circuit using special orthonormal gates with $d = 4$ layers approximating real-valued Haar random states. These parameters correspond to the Gompertz functions shown as dashed lines in figure 6.

Setting	α	β	γ
Random	1.002 ± 0.000	42.9 ± 0.2	0.538 ± 0.001
$d = 1$	0.084 ± 0.001	7.4 ± 0.3	0.252 ± 0.007
$d = 2$	0.048 ± 0.002	54.3 ± 7.9	0.386 ± 0.021
$d = 3$	0.040 ± 0.002	77.5 ± 6.9	0.387 ± 0.013
$d = 4$	0.035 ± 0.001	98.8 ± 5.6	0.389 ± 0.008

Table 3. Parameters obtained from fitting an algebraic decay $f(x) = \alpha x^{-\beta}$ as a function of the T -gate count to the infidelities between the circuits compiled into Clifford and T gates and the uncompressed states encoding the image data. We show results for states encoding images from the four datasets MNIST, Fashion-MNIST, CIFAR-10 and Imagenette, and for circuits using sparse, special orthogonal and general unitary gates pre-compilation. These parameters correspond to the algebraic decays shown as dashed lines in the upper half of figure 14.

Setting	α	β
MNIST		
Sparse	69.74 ± 33.21	0.809 ± 0.060
SO(4)	3222.11 ± 367.78	1.349 ± 0.013
U(4)	2919.40 ± 476.45	1.293 ± 0.018
Fashion-MNIST		
Sparse	89.28 ± 32.44	0.905 ± 0.046
SO(4)	5330.57 ± 924.25	1.466 ± 0.020
U(4)	4982.32 ± 878.37	1.413 ± 0.020
CIFAR-10		
Sparse	1.32 ± 0.13	0.418 ± 0.012
SO(4)	56.86 ± 4.98	0.949 ± 0.010
U(4)	56.01 ± 3.27	0.915 ± 0.006
Imagenette		
Sparse	1.51 ± 0.32	0.401 ± 0.028
SO(4)	7.15 ± 1.31	0.619 ± 0.020
U(4)	4.84 ± 0.65	0.555 ± 0.014

Table 4. Parameters obtained from fitting an algebraic growth $f(x) = \alpha x^\beta$ as a function of the T -gate count to the infidelities between the circuits compiled into Clifford and T gates and the same circuits pre-compilation. We show results for states encoding images from the four datasets MNIST, Fashion-MNIST, CIFAR-10 and Imagenette, and for circuits using sparse, special orthogonal and general unitary gates. These parameters correspond to the algebraic growths shown as dashed lines in the lower half of figure 14.

Setting	$\alpha \cdot 10^8$	β
MNIST		
Sparse	1.16 ± 0.16	1.03 ± 0.017
SO(4)	1.75 ± 0.08	0.99 ± 0.005
SU(4)	1.60 ± 0.07	1.00 ± 0.005
Fashion-MNIST		
Sparse	1.68 ± 0.17	0.99 ± 0.013
SO(4)	2.22 ± 0.19	0.96 ± 0.010
SU(4)	1.44 ± 0.14	1.01 ± 0.011
CIFAR-10		
Sparse	3.51 ± 0.31	0.90 ± 0.011
SO(4)	1.95 ± 0.36	0.97 ± 0.021
SU(4)	1.33 ± 0.22	1.02 ± 0.018
Imagenette		
Sparse	3.09 ± 0.85	0.92 ± 0.036
SO(4)	2.00 ± 0.68	0.97 ± 0.038
SU(4)	1.70 ± 0.12	0.99 ± 0.007

Appendix D. Details of the tensor-network classifiers

Here we give some more details on the tensor-network classifiers used in the main text. First, we show that the random initialization we use indeed leads to an output with unit variance. Then, we connect the MPO classifiers with the usual variational quantum classifiers that compute the expectation value of some Hermitian observable.

D.1. Numerically stable random initialization

In the main text we presented a method to randomly initialize the entries of an MPS classifier, and claimed that this ensured that the variance of the output of MPS classifier had unit variance. Here we are going to prove this claim for both the MPS and MPO classifier.

We consider an MPS classifier as shown in figure 9 in the main text. The MPS classifier has L sites, with the tensor on site i having an additional leg for the label index ℓ . We refer to the bond dimension on the m th bond as χ_m with $m = 1, \dots, L-1$ and consider arbitrary physical dimensions d_m on each site m . We will denote the entries of the MPS tensors $M_{\alpha_m \beta_m}^{j_m}$ on sites $m = 1, \dots, i-1, i+1, \dots, L$ and $M_{\alpha_i \beta_i}^{j_i \ell}$ on site i , where α_m, β_m are the virtual indices running over the local bond dimension, j_m is the physical index labeling the basis of the local d_m -dimensional Hilbert space and ℓ is the label index. For the initialization of these tensors, we assume that all entries are drawn independently from a Gaussian distribution with zero mean and a site-dependent variance σ_m^2 , $m = 1, \dots, L$. Further, we consider the case where the input state $\psi_{j_1, \dots, j_L}$ that is contracted with the MPS classifier is real and normalized as $\sum_{j_1, \dots, j_L} \psi_{j_1, \dots, j_L}^2 = 1$. For the MPS classifier, the input state is the normalized pure quantum state $\psi_{j_1, \dots, j_L} = \langle j_1, \dots, j_L | \psi(\mathbf{x}) \rangle$ encoding the data. Note that we can treat the MPO classifier in the same framework, by combining the physical indices k_m, k'_m of the MPO tensor into one index $j_m = (k_m, k'_m)$ with twice the size. Then, also the input state, which corresponds to the density matrix $\psi_{j_1, \dots, j_L} = \langle k_1, \dots, k_L | \rho(\mathbf{x}) | k'_1, \dots, k'_L \rangle = \langle k_1, \dots, k_L | \psi(\mathbf{x}) \rangle \langle \psi(\mathbf{x}) | k'_1, \dots, k'_L \rangle$ of the encoded data, is properly normalized since the density matrix corresponds to a pure state:

$$\begin{aligned} \sum_{j_1, \dots, j_L} \psi_{j_1, \dots, j_L}^2 &= \sum_{k_1, \dots, k_L} \langle k_1, \dots, k_L | \psi(\mathbf{x}) \rangle^2 \\ &\times \sum_{k'_1, \dots, k'_L} \langle \psi(\mathbf{x}) | k'_1, \dots, k'_L \rangle^2 = 1. \end{aligned} \quad (18)$$

Thus, in the following we can focus on the MPS classifier but all results also follow for the MPO classifier, which is essentially an MPS classifier with a doubled dimension of the physical Hilbert space.

Before computing the expectation value and variance of the MPS classifier contracted with an input state, we recall some simple identities for the expectation value and variance of sums and products of random variables. First, the expectation value is a linear function, so for a linear combination of multiple random variables X_n we have

$$\mathbb{E} \left[\sum_n a_n X_n \right] = \sum_n a_n \mathbb{E} [X_n]. \quad (19)$$

For independent random variables, the expectation value of a product becomes

$$\mathbb{E} \left[\prod_n X_n \right] = \prod_n \mathbb{E} [X_n]. \quad (20)$$

The variance of a sum of independent random variables behaves as

$$\text{Var} \left[\sum_n a_n X_n \right] = \sum_n a_n^2 \text{Var} [X_n], \quad (21)$$

and for the product of independent random variables which also have zero mean the variance is

$$\text{Var} \left[\prod_n X_n \right] = \prod_n \text{Var} [X_n]. \quad (22)$$

In our case, the random variables are the entries of the MPS tensors $M_{\alpha_m \beta_m}^{j_m}$ and $M_{\alpha_i \beta_i}^{j_i \ell}$, which are indeed sampled independently and have zero mean, so we can make use of the relations above.

We denote the result of the contraction of the MPS classifier with an input state as $f_\ell(\mathbf{x})$ —the decision function of the model. Then, for the expectation value we have

$$\begin{aligned} \mathbb{E} [f_\ell(\mathbf{x})] &= \mathbb{E} \left[\sum_{j_1, \dots, j_L} \sum_{\substack{\alpha_2, \dots, \alpha_L \\ \beta_1, \dots, \beta_{L-1}}} \psi_{j_1, \dots, j_L} \left(\prod_{k=1}^{L-1} \delta_{\beta_k, \alpha_{k+1}} \right) \left(\prod_{m=1}^{i-1} M_{\alpha_m, \beta_m}^{j_m} \right) M_{\alpha_i \beta_i}^{j_i \ell} \left(\prod_{n=i+1}^L M_{\alpha_n, \beta_n}^{j_n} \right) \right] \\ &= \sum_{j_1, \dots, j_L} \sum_{\substack{\alpha_2, \dots, \alpha_L \\ \beta_1, \dots, \beta_{L-1}}} \psi_{j_1, \dots, j_L} \left(\prod_{k=1}^{L-1} \delta_{\beta_k, \alpha_{k+1}} \right) \mathbb{E} \left[\left(\prod_{m=1}^{i-1} M_{\alpha_m, \beta_m}^{j_m} \right) M_{\alpha_i \beta_i}^{j_i \ell} \left(\prod_{n=i+1}^L M_{\alpha_n, \beta_n}^{j_n} \right) \right] \\ &= \sum_{j_1, \dots, j_L} \sum_{\substack{\alpha_2, \dots, \alpha_L \\ \beta_1, \dots, \beta_{L-1}}} \psi_{j_1, \dots, j_L} \left(\prod_{k=1}^{L-1} \delta_{\beta_k, \alpha_{k+1}} \right) \left(\prod_{m=1}^{i-1} \mathbb{E} [M_{\alpha_m, \beta_m}^{j_m}] \right) \mathbb{E} [M_{\alpha_i \beta_i}^{j_i \ell}] \left(\prod_{n=i+1}^L \mathbb{E} [M_{\alpha_n, \beta_n}^{j_n}] \right) \\ &= 0. \end{aligned} \quad (23)$$

The first line simply writes out the expression for contracting the MPS classifier with its input state. To get to the second line we used the linearity of the expectation value; to get to the third line we used the independence of the random entries of the MPS tensors; and, for the final result, we used that the random variables have zero mean.

For the variance of the decision function $f_\ell(\mathbf{x})$ we have

$$\begin{aligned} \text{Var} [f_\ell(\mathbf{x})] &= \text{Var} \left[\sum_{j_1, \dots, j_L} \sum_{\substack{\alpha_2, \dots, \alpha_L \\ \beta_1, \dots, \beta_{L-1}}} \psi_{j_1, \dots, j_L} \left(\prod_{k=1}^{L-1} \delta_{\beta_k, \alpha_{k+1}} \right) \left(\prod_{m=1}^{i-1} M_{\alpha_m, \beta_m}^{j_m} \right) M_{\alpha_i \beta_i}^{j_i \ell} \left(\prod_{n=i+1}^L M_{\alpha_n, \beta_n}^{j_n} \right) \right] \\ &= \sum_{j_1, \dots, j_L} \sum_{\substack{\alpha_2, \dots, \alpha_L \\ \beta_1, \dots, \beta_{L-1}}} \psi_{j_1, \dots, j_L}^2 \left(\prod_{k=1}^{L-1} \delta_{\beta_k, \alpha_{k+1}} \right) \text{Var} \left[\left(\prod_{m=1}^{i-1} M_{\alpha_m, \beta_m}^{j_m} \right) M_{\alpha_i \beta_i}^{j_i \ell} \left(\prod_{n=i+1}^L M_{\alpha_n, \beta_n}^{j_n} \right) \right] \\ &= \sum_{j_1, \dots, j_L} \sum_{\substack{\alpha_2, \dots, \alpha_L \\ \beta_1, \dots, \beta_{L-1}}} \psi_{j_1, \dots, j_L}^2 \left(\prod_{k=1}^{L-1} \delta_{\beta_k, \alpha_{k+1}} \right) \left(\prod_{m=1}^{i-1} \text{Var} [M_{\alpha_m, \beta_m}^{j_m}] \right) \text{Var} [M_{\alpha_i \beta_i}^{j_i \ell}] \left(\prod_{n=i+1}^L \text{Var} [M_{\alpha_n, \beta_n}^{j_n}] \right) \\ &= \sum_{j_1, \dots, j_L} \psi_{j_1, \dots, j_L}^2 \sum_{\substack{\alpha_2, \dots, \alpha_L \\ \beta_1, \dots, \beta_{L-1}}} \left(\prod_{k=1}^{L-1} \delta_{\beta_k, \alpha_{k+1}} \right) \left(\prod_{m=1}^L \sigma_m^2 \right), \end{aligned} \quad (24)$$

where the first line again just writes out the expression for the contraction of the MPS; to get to the second line we made use of the sum-rule for variances in equation (21) (note that $\delta_{\alpha_k, \beta_{k+1}}^2 = \delta_{\alpha_k, \beta_{k+1}}$); to get to the third line we used the product-rule for independent random variables with zero mean in equation (22); and in the last line we plugged in the variances of the random entries of the MPS tensors. To simplify the result further, we can use that the input state $\psi_{j_1, \dots, j_L}$ is normalized and we can get rid of the sums over β_k by making use of the Kronecker deltas. This yields

$$\begin{aligned} \text{Var}[f_\ell(\mathbf{x})] &= \sum_{\alpha_2, \dots, \alpha_L} \left(\prod_{m=1}^L \sigma_m^2 \right) \\ &= \left(\prod_{n=1}^{L-1} \chi_n \right) \left(\prod_{m=1}^L \sigma_m^2 \right), \end{aligned} \quad (25)$$

where χ_n is the bond dimension on the n th bond.

To ensure that the final output has unit variance, we can rescale the standard deviations of the Gaussian distributions used for initializing the MPS tensors as described in the main text. For MPS tensors left of the one containing the label index ℓ , we choose the standard deviation as $\sigma_m = 1/\sqrt{\chi_{m-1}}$ where χ_{m-1} is the bond dimension of the bond left of the tensor and we set $\chi_0 = 1$. For MPS tensors right of the label index ℓ , we choose the standard deviations as $\sigma_m = 1/\sqrt{\chi_m}$, where χ_m is the bond dimension of the bond right of the tensor and we set $\chi_L = 1$. The tensor containing the label index ℓ is initialized with standard deviation $\sigma_i = 1/\sqrt{\chi_{i-1}\chi_i}$. Plugging these choices for the standard deviations into the result for the variance above, we obtain

$$\begin{aligned} \text{Var}[f_\ell(\mathbf{x})] &= \left(\prod_{k=1}^{L-1} \chi_k \right) \left(\prod_{m=1}^{i-1} \frac{1}{\chi_{m-1}} \right) \frac{1}{\chi_{i-1}\chi_i} \left(\prod_{n=i+1}^L \frac{1}{\chi_n} \right) \\ &= 1. \end{aligned} \quad (26)$$

Hence, the output has unit variance as claimed in the main text.

D.2. MPO classifiers are a subset of general quantum classifiers

In the main text we have introduced the MPO classifier because its decision function is a linear function with respect to the input density matrix, just like a quantum circuit classifier. However, the expectation values of the quantum circuit classifiers are usually taken with respect to a Hermitian observable, and the operator implemented by the MPO is not constrained to be Hermitian. Here we show that only the Hermitian part of the MPO contributes to the output, so the decision function of the MPO classifier can be viewed as the expectation value of a Hermitian observable—making the MPO classifier a proper subset of the most general quantum circuit classifiers.

Let us denote the operator implemented by the MPO classifier as O_ℓ , where the index ℓ is the label index and corresponds to the one additional leg of the MPO that is not contracted with the input state. The operator O_ℓ is purely real, as all MPO tensors are real, and we can split it into a symmetric part $O_\ell^+ = \frac{1}{2}(O_\ell + O_\ell^T)$ (which is also Hermitian) and an anti-symmetric part $O_\ell^- = \frac{1}{2}(O_\ell - O_\ell^T)$. We recover the original operator as $O_\ell = O_\ell^+ + O_\ell^-$. For the expectation value of the anti-symmetric part with respect to a purely real-valued input state $|\psi(\mathbf{x})\rangle$ we have

$$\begin{aligned} \langle \psi(\mathbf{x}) | O_\ell^- | \psi(\mathbf{x}) \rangle &= (\langle \psi(\mathbf{x}) | O_\ell^- | \psi(\mathbf{x}) \rangle)^T \\ &= \langle \psi(\mathbf{x}) | (O_\ell^-)^T | \psi(\mathbf{x}) \rangle \\ &= -\langle \psi(\mathbf{x}) | O_\ell^- | \psi(\mathbf{x}) \rangle \end{aligned} \quad (27)$$

and thus $\langle \psi(\mathbf{x}) | O_\ell^- | \psi(\mathbf{x}) \rangle = 0$. Therefore, the expectation value of the full operator O_ℓ is

$$\begin{aligned} \langle \psi(\mathbf{x}) | O_\ell | \psi(\mathbf{x}) \rangle &= \langle \psi(\mathbf{x}) | O_\ell^+ | \psi(\mathbf{x}) \rangle + \langle \psi(\mathbf{x}) | O_\ell^- | \psi(\mathbf{x}) \rangle \\ &= \langle \psi(\mathbf{x}) | O_\ell^+ | \psi(\mathbf{x}) \rangle. \end{aligned} \quad (28)$$

Since $O_\ell^+ = (O_\ell^+)^T$ is a Hermitian operator, the decision function of the MPO classifier can be written as the expectation value of a Hermitian observable as claimed.

ORCID iDs

Florian J Kiwit  0009-0000-4065-1535
 Bernhard Jobst  0000-0001-7027-3918
 Andre Luckow  0000-0002-1225-4062
 Frank Pollmann  0000-0003-0320-9304
 Carlos A Riofrío  0000-0002-7346-9198

References

- [1] Schuld M and Petruccione F 2022 *Machine Learning With Quantum Computers* 2nd edn (Springer)
- [2] Biamonte J, Wittek P, Pancotti N, Rebentrost P, Wiebe N and Lloyd S 2017 Quantum machine learning *Nature* **549** 195–202
- [3] Liu Y, Arunachalam S and Temme K 2021 A rigorous and robust quantum speed-up in supervised machine learning *Nat. Phys.* **17** 1013–7
- [4] Sweke R, Seifert J-P, Hangleiter D and Eisert J 2021 On the quantum versus classical learnability of discrete distributions *Quantum* **5** 417
- [5] Huang H-Y, Broughton M, Mohseni M, Babbush R, Boixo S, Neven H and McClean J R 2021 Power of data in quantum machine learning *Nat. Commun.* **12** 2631
- [6] Pirnay N, Sweke R, Eisert J and Seifert J-P 2023 Superpolynomial quantum-classical separation for density modeling *Phys. Rev. A* **107** 042416
- [7] Gyurik C and Dunjko V 2023 Exponential separations between classical and quantum learners (arXiv:2306.16028)
- [8] Gil-Fuster E, Gyurik C, Pérez-Salinas A and Dunjko V 2024 On the relation between trainability and dequantization of variational quantum learning models (arXiv:2406.07072)
- [9] Zimborás Z *et al* 2025 Myths around quantum computation before full fault tolerance: what no-go theorems rule out and what they don't (arXiv:2501.05694)
- [10] Stoudenmire E M and Schwab D J 2016 Supervised learning with quantum-inspired tensor networks *Advances in Neural Information Processing Systems* vol 29, ed D Lee, M Sugiyama, U Luxburg, I Guyon and R Garnett (Curran Associates, Inc.) pp 4799–807 (available at: https://proceedings.neurips.cc/paper_files/paper/2016/file/5314b9674c86c3f9d1ba25ef9bb32895-paper.pdf)
- [11] Pérez-Salinas A, Cervera-Lierta A, Gil-Fuster E and Latorre J I 2020 Data re-uploading for a universal quantum classifier *Quantum* **4** 226
- [12] Goto T, Tran Q H and Nakajima K 2021 universal approximation property of quantum machine learning models in quantum-enhanced feature spaces *Phys. Rev. Lett.* **127** 090506
- [13] Schuld M, Sweke R and Meyer J J 2021 Effect of data encoding on the expressive power of variational quantum-machine-learning models *Phys. Rev. A* **103** 032430
- [14] Latorre J I 2005 Image compression and entanglement (arXiv:quant-ph/0510031)
- [15] Plesch M and Brukner Časlav 2011 Quantum-state preparation with universal gate decompositions *Phys. Rev. A* **83** 032302
- [16] Iten R, Colbeck R, Kukuljan I, Home J and Christandl M 2016 Quantum circuits for isometries *Phys. Rev. A* **93** 032318
- [17] Grant E, Benedetti M, Cao S, Hallam A, Lockhart J, Stojevic V, Green A G and Severini S 2018 Hierarchical quantum classifiers *npj Quantum Inf.* **4** 65
- [18] Havlíček V, Córcoles A D, Temme K, Harrow A W, Kandala A, Chow J M and Gambetta J M 2019 Supervised learning with quantum-enhanced feature spaces *Nature* **567** 209–12
- [19] Schuld M and Killoran N 2019 Quantum machine learning in feature Hilbert spaces *Phys. Rev. Lett.* **122** 040504
- [20] Schuld M, Bocharov A, Svore K M and Wiebe N 2020 Circuit-centric quantum classifiers *Phys. Rev. A* **101** 032308
- [21] Bartkiewicz K, Gneiting C, Černoch A, Jiráková K, Lemr K and Nori F 2020 Experimental kernel-based quantum machine learning in finite feature space *Sci. Rep.* **10** 12356
- [22] Kerenidis I and Luongo A 2020 Quantum classification of the MNIST dataset with Slow Feature Analysis *Phys. Rev. A* **101** 062327
- [23] Chalumuri A, Kune R and Manoj B S 2021 A hybrid classical-quantum approach for multi-class classification *Quantum Inf. Process.* **20** 119
- [24] Johri S, Debnath S, Mocherla A, Singk A, Prakash A, Kim J and Kerenidis I 2021 Nearest centroid classification on a trapped ion quantum computer *npj Quantum Inf.* **7** 1–11
- [25] Peters E, Caldeira J ao, Ho A, Leichenauer S, Mohseni M, Neven H, Spentzouris P, Strain D and Perdue G N 2021 Machine learning of high dimensional data on a noisy quantum processor *npj Quantum Inf.* **7** 1–5
- [26] Bokhan D, Mastiukova A S, Boev A S, Trubnikov D N and Fedorov A K 2022 Multiclass classification using quantum convolutional neural networks with hybrid quantum-classical learning *Front. Phys.* **10** 1069985
- [27] Hur T, Kim L and Park D K 2022 Quantum convolutional neural network for classical data classification *Quantum Mach. Intell.* **4** 3
- [28] Lu J Z, Jiao L, Wolinski K, Kornjača M, Hu H-Y, Cantu S, Liu F, Yelin S F and Wang S-T 2024 Digital-analog quantum learning on Rydberg atom arrays *Quantum Sci. Technol.* **10** 015038
- [29] Bowles J, Ahmed S and Schuld M 2024 Better than classical? The subtle art of benchmarking quantum machine learning models (arXiv:2403.07059)
- [30] Zhou N-R, Liu X-X, Chen Y-L and Du N-S 2021 Quantum k-nearest-neighbor image classification algorithm based on K-L transform *Int. J. Theor. Phys.* **60** 1209–24
- [31] Riaz F, Abdulla S, Suzuki H, Ganguly S, Deo R C and Hopkins S 2023 Accurate image multi-class classification neural network model with quantum entanglement approach *Sensors* **23** 2753
- [32] Baek H, Park S and Kim J 2023 Logarithmic dimension reduction for quantum neural networks *Proc. 32nd ACM Int. Conf. on Information and Knowledge Management (CIKM '23)* (Association for Computing Machinery) pp 3738–42
- [33] Khatun A and Usman M 2024 Quantum transfer learning with adversarial robustness for classification of high-resolution image datasets *Adv. Quantum Technol.* **8** 2400268

- [34] Monbroussou L, Landman J, Wang L, Grilo A B and Kashefi E 2024 Subspace preserving quantum convolutional neural network architectures (arXiv:2409.18918)
- [35] Russakovsky O et al 2015 ImageNet large scale visual recognition challenge (arXiv:1409.0575)
- [36] Krizhevsky A, Sutskever I and Hinton G E 2012 ImageNet classification with deep convolutional neural networks *Advances in Neural Information Processing Systems* vol 25, ed F Pereira, C J Burges, L Bottou and K Q Weinberger (Curran Associates, Inc.) (available at: https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-paper.pdf)
- [37] Moussa C, van Rijn J N, Bäck T and Dunjko V 2022 *Hyperparameter Importance of Quantum Neural Networks Across Small Datasets* (Springer) pp 32–46
- [38] Kashif M, Rashid M, Al-Kuwari S and Shafique M 2023 Alleviating barren plateaus in parameterized quantum machine learning circuits: Investigating advanced parameter initialization strategies (arXiv:2311.13218)
- [39] Finžgar J R, Ross P, Holscher L, Klepsch J and Luckow A 2022 QUARK: a framework for quantum computing application benchmarking 2022 *IEEE Int. Conf. on Quantum Computing and Engineering (QCE)* (IEEE)
- [40] Kiwit F J, Marso M, Ross P, Riofrío C A, Klepsch J and Luckow A 2023 Application-oriented benchmarking of quantum generative learning using QUARK 2023 *IEEE Int. Conf. on Quantum Computing and Engineering (QCE)* vol 01 pp 475–84
- [41] Kiwit F J, Wolf M A, Marso M, Ross P, Lorenz J M, Riofrío C A and Luckow A 2024 Benchmarking quantum generative learning: a study on scalability and noise resilience using QUARK *KI - Künstliche Intell.* **38** 379–85
- [42] Quetschlich N, Burgholzer L and Wille R 2023 MQT Bench: Benchmarking software and design automation tools for quantum computing *Quantum* **7** 1062
- [43] Dilip R, Liu Y-J, Smith A and Pollmann F 2022 Data compression for quantum machine learning *Phys. Rev. Res.* **4** 043007
- [44] Jobst B, Shen K, Riofrío C A, Shishenina E and Pollmann F 2024 Efficient MPS representations and quantum circuits from the Fourier modes of classical image data *Quantum* **8** 1544
- [45] Iaconis J and Johri S 2023 Tensor network based efficient quantum data loading of images (arXiv:2310.05897)
- [46] Shen K, Jobst B, Shishenina E and Pollmann F 2024 Classification of the Fashion-MNIST dataset on a quantum computer (arXiv:2403.02405)
- [47] West M T, Nakhil A C, Heredge J, Creevey F M, Hollenberg L C L, Sevier M and Usman M 2024 Drastic circuit depth reductions with preserved adversarial robustness by approximate encoding for quantum machine learning *Intell. Comput.* **3** 0100
- [48] Lecun Y, Bottou L, Bengio Y and Haffner P 1998 Gradient-based learning applied to document recognition *Proc. IEEE* **86** 2278–324
- [49] Deng Li 2012 The MNIST database of handwritten digit images for machine learning research *IEEE Signal Process. Mag.* **29** 141–2
- [50] Xiao H, Rasul K and Vollgraf R 2017 Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms (arXiv:1708.07747)
- [51] Krizhevsky A 2009 Learning multiple layers of features from tiny images (available at: www.cs.toronto.edu/kriz/learning-features-2009-TR.pdf)
- [52] Imagenette J H 2019 (available at: <https://github.com/fastai/imagenette>)
- [53] Bergholm V et al 2022 PennyLane: automatic differentiation of hybrid quantum-classical computations (arXiv:1811.04968)
- [54] Lubasch M, Moinier P and Jaksch D 2018 Multigrid renormalization *J. Comput. Phys.* **372** 587–602
- [55] Lubasch M, Joo J, Moinier P, Kiffner M and Jaksch D 2020 Variational quantum algorithms for nonlinear problems *Phys. Rev. A* **101** 010301
- [56] Gourianov N, Lubasch M, Dolgov S, van den Berg Q Y, Babaee H, Givi P, Kiffner M and Jaksch D 2022 A quantum-inspired approach to exploit turbulence structures *Nat. Comput. Sci.* **2** 30–37
- [57] Holscher L, Rao P, Müller L, Klepsch J, Luckow A, Stollenwerk T and Wilhelm F K 2025 Quantum-inspired fluid simulation of two-dimensional turbulence with GPU acceleration *Phys. Rev. Res.* **7** 013112
- [58] Ritter M K, Fernández Y N, Wallerberger M, von Delft J, Shinaoka H and Waintal X 2024 Quantics tensor cross interpolation for high-resolution parsimonious representations of multivariate functions *Phys. Rev. Lett.* **132** 056501
- [59] Amankwah M G, Camps D, Wes Bethel E, Van Beeumen R and Perciano T 2022 Quantum pixel representations and compression for N -dimensional images *Sci. Rep.* **12** 7712
- [60] Le P Q, Dong F and Hirota K 2011 A flexible representation of quantum images for polynomial preparation, image compression and processing operations *Quantum Inf. Process.* **10** 63–84
- [61] Le P Q, Ilyasu A M, Dong F and Hirota K 2011 *A Flexible Representation and Invertible Transformations for Images on Quantum Computers* vol 372 (Springer) pp 179–202
- [62] Savostyanov D V and Oseledets I 2011 Fast adaptive interpolation of multi-dimensional arrays in tensor train format *The 2011 Int. Workshop on Multidimensional (nD) Systems* pp 1–8
- [63] Savostyanov D V 2014 Quasioptimality of maximum-volume cross interpolation of tensors *Linear Algebr. Appl.* **458** 217–44
- [64] Dolgov S and Dmitry V S 2020 Parallel cross interpolation for high-precision calculation of high-dimensional integrals *Comput. Phys. Commun.* **246** 106869
- [65] Núñez Fernández Y, Jeannin M, Dumitrescu P T, Kloss T, Kaye J, Parcollet O and Waintal X 2022 Learning Feynman diagrams with tensor trains *Phys. Rev. X* **12** 041018
- [66] Núñez Fernández Y et al 2024 Learning tensor networks with tensor cross interpolation: new algorithms and libraries (arXiv:2407.02454)
- [67] Sun B, Le P Q, Ilyasu A M, Yan F, Adrian Garcia J, Dong F and Hirota K 2011 A multi-channel representation for images on quantum computers using the RGB α color space 2011 *IEEE 7th Int. Symp. on Intelligent Signal Processing* pp 1–6
- [68] Sun B, Ilyasu A M, Yan F, Dong F and Hirota K 2013 An RGB multi-channel representation for images on quantum computers *J. Adv. Comput. Intell. Intell. Inf.* **17** 404–17
- [69] Schuld M 2024 Supervised quantum machine learning models are kernel methods (arXiv:2101.11020)
- [70] Holmes Z, Coble N J, Sornborger A T and Subaşı Y 2023 Nonlinear transformations in quantum computation *Phys. Rev. Res.* **5** 013105
- [71] Schön C, Solano E, Verstraete F, Ignacio Cirac J and Wolf M M 2005 Sequential generation of entangled multiqubit states *Phys. Rev. Lett.* **95** 110503
- [72] Schön C, Hammerer K, Wolf M M, Cirac J I and Solano E 2007 Sequential generation of matrix-product states in cavity QED *Phys. Rev. A* **75** 032311
- [73] Smith A, Jobst B, Green A G and Pollmann F 2022 Crossing a topological phase transition with a quantum computer *Phys. Rev. Res.* **4** L022020

- [74] Lin S-H, Dilip R, Green A G, Smith A and Pollmann F 2021 Real- and imaginary-time evolution with compressed quantum circuits *PRX Quantum* **2** 010342
- [75] Barratt F, Dborin J, Bal M, Stojevic V, Pollmann F and Green A G 2021 Parallel quantum simulation of large systems on small NISQ computers *npj Quantum Inf.* **7** 79
- [76] Ran S-J 2020 Encoding of matrix product states into quantum circuits of one- and two-qubit gates *Phys. Rev. A* **101** 032310
- [77] Rudolph M S, Chen J, Miller J, Acharya A and Perdomo-Ortiz A 2023 Decomposition of matrix product states into shallow quantum circuits *Quantum Sci. Technol.* **9** 015012
- [78] Ben-Dov M, Shnaiderov D, Makmal A and Dalla Torre E G 2024 Approximate encoding of quantum states using shallow circuits *npj Quantum Inf.* **10** 65
- [79] Schollwöck U 2011 The density-matrix renormalization group in the age of matrix product states *Ann. Phys., NY* **326** 96–192
- [80] Orús R 2014 A practical introduction to tensor networks: Matrix product states and projected entangled pair states *Ann. Phys., NY* **349** 117–58
- [81] Wei Z-Y, Malz D and Cirac J I 2022 Sequential generation of projected entangled-pair states *Phys. Rev. Lett.* **128** 010607
- [82] Bohun V, Lukin I, Luhanko M, Korpas G, De Brouwer P J S, Maksymenko M and Koch-Janusz M 2024 Scalable and shallow quantum circuits encoding probability distributions informed by asymptotic entanglement analysis (arXiv:2412.05202)
- [83] Shende V V, Bullock S S and Markov I L 2004 Recognizing small-circuit structure in two-qubit operators *Phys. Rev. A* **70** 012310
- [84] Shende V V, Markov I L and Bullock S S 2004 Minimal universal two-qubit controlled-NOT-based circuits *Phys. Rev. A* **69** 062321
- [85] Wei H-R and Di. Y-M 2012 Decomposition of orthogonal matrix and synthesis of two-qubit and three-qubit orthogonal gates *Quant. Inf. Comput.* **12** 0262–70
- [86] Evenbly G and Vidal G 2009 Algorithms for entanglement renormalization *Phys. Rev. B* **79** 144108
- [87] Nocedal J and Stephen J W 2006 *Numerical Optimization* 2nd edn (Springer)
- [88] Hauru M, Van Damme M and Haegeman J 2021 Riemannian optimization of isometric tensor networks *SciPost Phys.* **10** 040
- [89] Dborin J, Barratt F, Wimalaweera V, Wright L and Green A G 2022 Matrix product state pre-training for quantum machine learning *Quantum Sci. Technol.* **7** 035014
- [90] Zhang K, Liu L, Hsieh M-H, Tao D Agarwal A 2022 Escaping from the barren plateau via gaussian initializations in deep variational quantum circuits *Advances in Neural Information Processing Systems* ed A H Oh, D Belgrave and K Cho (available at: <https://openreview.net/forum?id=jXgbJdQ2YIy>)
- [91] Rudolph M S, Miller J, Motlagh D, Chen J, Acharya A and Perdomo-Ortiz A 2023 Synergistic pretraining of parametrized quantum circuits via tensor networks *Nat. Commun.* **14** 8367
- [92] Park C-Y and Killoran N 2024 Hamiltonian variational ansatz without barren plateaus *Quantum* **8** 1239
- [93] Wang Y, Qi B, Ferrie C and Dong D 2024 Trainability enhancement of parameterized quantum circuits via reduced-domain parameter initialization *Phys. Rev. Appl.* **22** 054005
- [94] Puig R, Drudis M, Thanasilp S and Holmes Z 2025 Variational quantum simulation: A case study for understanding warm starts *PRX Quantum* **6** 010317
- [95] Mhiri H, Puig R, Lerch S, Rudolph M S, Chotibut T, Thanasilp S and Holmes Z 2025 A unifying account of warm start guarantees for patches of quantum landscapes (arXiv:2502.07889)
- [96] Higham N J 1989 Matrix nearness problems and applications *Applications of Matrix Theory* ed M J C Gover and S Barnett (Oxford University Press) pp 1–27
- [97] Myronenko A and Song X 2009 On the closed-form solution of the rotation matrix arising in computer vision problems (arXiv:0904.1613)
- [98] Moritz P et al 2018 Ray: a distributed framework for emerging ai applications *Proc. 13th USENIX Conf. on Operating Systems Design and Implementation (OSDI'18)* (USENIX Association) pp 561–77 (available at: www.usenix.org/conference/osdi18/presentation/moritz)
- [99] Bradbury J, Frostig R, Hawkins P, Johnson M J, Leary C, Maclaurin D, Necula G, Paszke A, VanderPlas J, Wanderman-Milne S and Zhang Q 2025 JAX: composable transformations of Python+NumPy programs (available at: <https://github.com/google/jax>)
- [100] Virtanen P et al 2020 SciPy 1.0: fundamental algorithms for scientific computing in python *Nat. Methods* **17** 261–72
- [101] Ciregan D, Meier U and Schmidhuber J 2012 Multi-column deep neural networks for image classification *2012 IEEE Conf. on Computer Vision and Pattern Recognition* pp 3642–9
- [102] Wan Li, Zeiler M, Zhang S, Cun Y L and Fergus R 2013 Regularization of neural networks using dropconnect *Proc. 30th Int. Conf. on Machine Learning (Proc. of Machine Learning Research)* vol 28, ed S Dasgupta and D McAllester (PMLR) pp 1058–66 (available at: <https://proceedings.mlr.press/v28/wan13.html>)
- [103] Hasanpour S H, Rouhani M, Fayyaz M and Sabokrou M 2016 Lets keep it simple, using simple architectures to outperform deeper and more complex architectures (arXiv:1608.06037)
- [104] Rajasegaran J, Jayasundara V, Jayasekara S, Jayasekara H, Seneviratne S and Rodrigo R 2019 DeepCaps: going deeper with capsule networks *2019 IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)* pp 10717–25
- [105] Meshkini K, Platos J and Ghassemain H 2020 An analysis of convolutional neural network for fashion images classification (Fashion-MNIST) *Proc. 14th Int. Scientific Conf. "Intelligent Information Technologies for Industry (IITI)"* vol 19, ed S Kovalev, V Tarassov, V Snasel and A Sukhanov (Springer) pp 85–95
- [106] Kaye M, Anter A and Mohamed H 2020 Classification of garments from Fashion MNIST dataset using CNN LeNet-5 architecture *2020 Int. Conf. on Innovative Trends in Communication and Computer Engineering (ITCE)* pp 238–43
- [107] Huang G, Liu Z, Maaten L V D and Weinberger K Q 2017 Densely connected convolutional networks *2017 IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)* pp 2261–9
- [108] Deng J, Dong W, Socher R, Li Li-J, Li K and Fei-Fei Li 2009 ImageNet: a large-scale hierarchical image database *2009 IEEE Conf. on Computer Vision and Pattern Recognition* pp 248–55 (available at: www.image-net.org)
- [109] Tange O 2024 GNU Parallel 20240922 ('Gold Apollo AR924') (available at: <https://doi.org/10.5281/zenodo.13826092>)
- [110] Mitarai K, Negoro M, Kitagawa M and Fujii K 2018 Quantum circuit learning *Phys. Rev. A* **98** 032309
- [111] Farhi E and Neven H 2018 Classification with quantum neural networks on near term processors (arXiv:1802.06002)
- [112] Liu Z, Yu L-W, Duan L-M and Deng D-L 2022 Presence and absence of barren plateaus in tensor-network based machine learning *Phys. Rev. Lett.* **129** 270501
- [113] Zhang H-K, Liu S and Zhang S-X 2024 Absence of barren plateaus in finite local-depth circuits with long-range entanglement *Phys. Rev. Lett.* **132** 150603
- [114] Barthel T and Miao Q 2025 Absence of barren plateaus and scaling of gradients in the energy optimization of isometric tensor network states *Commun. Math. Phys.* **406** 86

- [115] Kingma D P and Jimmy B 2015 Adam: a method for stochastic optimization *3rd Int. Conf. on Learning Representations (ICLR) (San Diego, CA, USA)*
- [116] Hornik K 1991 Approximation capabilities of multilayer feedforward networks *Neural Netw.* **4** 251–7
- [117] Jerbi S, Fiderer L J, Nautrup H P, Kübler J M, Briegel H J and Dunjko V 2023 Quantum machine learning beyond kernel methods *Nat. Commun.* **14** 517
- [118] Schölkopf B and Smola A J 2001 *Learning With Kernels: Support Vector Machines, Regularization, Optimization and Beyond* (MIT Press)
- [119] Pedregosa F et al 2011 Scikit-learn: machine learning in Python *J. Mach.: Learn. Res.* **12** 2825–30 (available at: <https://jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf>)
- [120] Novikov A, Trofimov M and Oseledets I 2018 Exponential machines *Bull. Pol. Acad. Sci. Tech. Sci.* **66** 789–97
- [121] Huggins W, Patil P, Mitchell B, Birgitta Whaley K and Miles Stoudenmire E 2019 Towards quantum machine learning with tensor networks *Quantum Sci. Technol.* **4** 024001
- [122] Efthymiou S, Hidayi J and Leichenauer S 2019 Tensor network for machine learning (arXiv:1906.06329)
- [123] Barratt F, Dborin J and Wright L 2022 Improvements to gradient descent methods for quantum tensor network machine learning *2nd Workshop on Quantum Tensor Networks in Machine Learning, 35th Conf. on Neural Information Processing Systems (NeurIPS 2021)*
- [124] Lin S-H, Kuijpers O, Peterhans S and Pollmann F 2023 Distributive pre-training of generative modeling using matrix-product states *35th Conf. on Neural Information Processing Systems*
- [125] Nemkov N A, Kiktenko E O and Fedorov A K 2023 Fourier expansion in variational quantum algorithms *Phys. Rev. A* **108** 032406
- [126] Okumura S and Ohzeki M 2023 Fourier coefficient of parameterized quantum circuits and barren plateau problem (arXiv:2309.06740)
- [127] Barthe A and Pérez-Salinas A 2024 Gradients and frequency profiles of quantum re-uploading models *Quantum* **8** 1523
- [128] LeCun Y, Boser B, Denker J, Henderson D, Howard R, Hubbard W and Jackel L 1989 Handwritten digit recognition with a back-propagation network *Advances in Neural Information Processing Systems* vol 2, editor D Touretzky (Morgan-Kaufmann) (available at: https://proceedings.neurips.cc/paper_files/paper/1989/file/53c3bce66e43be4f209556518c2fcb54-paper.pdf)
- [129] Dosovitskiy A et al 2021 An image is worth 16 × 16 words: transformers for image recognition at scale *Int. Conf. on Learning Representations* (available at: <https://openreview.net/forum?id=YicbFdNTTy>)
- [130] Zeiler M D 2012 Adadelata: an adaptive learning rate method (arXiv:1212.5701)
- [131] Glorot X and Bengio Y 2010 Understanding the difficulty of training deep feedforward neural networks *Proc. 13th Int. Conf. on Artificial Intelligence and Statistics (Chia Laguna Resort, Sardinia, Italy, 13 May–15 May 2010) (Proc. of Machine Learning Research)* vol 9, ed Y W Teh and M Titterton (PMLR) pp 249–56 (available at: <https://proceedings.mlr.press/v9/glorot10a.html>)
- [132] Li L, Jamieson K, Rostamizadeh A, Gonina E, Ben-tzur J, Hardt M, Recht B and Talwalkar A 2020 A system for massively parallel hyperparameter tuning *Proc. of Machine Learning and Systems* vol 2, ed I Dhillon, D Papailiopoulos and V Sze pp 230–46 (available at: https://proceedings.mlsys.org/paper_files/paper/2020/file/a06f20b349c6cf09a6b171c71b88bbfc-paper.pdf)
- [133] Bergstra J, Yamins D and Cox D 2013 Making a science of model search: Hyperparameter optimization in hundreds of dimensions for vision architectures *Proc. 30th Int. Conf. on Machine Learning (Atlanta, Georgia, USA, 17 June–19 June 2013) (Proc. of Machine Learning Research)* vol 28, ed S Dasgupta and D McAllester (PMLR) pp 115–23 (available at: <https://proceedings.mlr.press/v28/bergstra13.html>)
- [134] Page D N 1993 Average entropy of a subsystem *Phys. Rev. Lett.* **71** 1291–4
- [135] Bravyi S and Kitaev A 2005 Universal quantum computation with ideal Clifford gates and noisy ancillas *Phys. Rev. A* **71** 022316
- [136] Ross N J and Selinger P 2016 Optimal ancilla-free Clifford+T approximation of Z-rotations *Quantum Inf. Comput.* **16** 0901–53
- [137] Yamamoto S and Yoshioka N 2024 Pygridsynth: a native python implementation of the gridsynth algorithm (available at: <https://github.com/quantum-programming/pygridsynth>)